

Camada de Autorização para Pagamentos Iniciados por Agentes

Briefing v4.0 — Avaliação crítica da v3.0, correções, camada de revogação em tempo real e arquitetura de implementação completa (MVA e production-grade)

Versão: 4.0 · **Data:** 29 de agosto de 2026 · **Base:** documento técnico v3.0 + pesquisa sobre revogação de mandatos AP2
Natureza: documento de engenharia para hackathon. Não constitui aconselhamento jurídico, regulatório ou financeiro.
Escopo desta revisão: 18 achados sobre a v3.0 (3 críticos, 6 altos, 8 médios, 1 de execução), resposta técnica às 14 perguntas da pesquisa de revogação, arquitetura de implementação em dois níveis, plano de execução hora a hora e roteiro de demonstração revisado.

COMO LER ESTE DOCUMENTO

A v3.0 continua sendo o documento de referência de *especificação*. Este briefing **não a substitui**: ele a audita, corrige o que está errado, preenche o que está ausente e acrescenta a camada que faltava — a de execução do pagamento e a de revogação em tempo real. Onde há conflito entre v3.0 e v4.0, prevalece a v4.0. As seções §0 a §2 avaliam; §3 corrige; §4 responde à pesquisa de revogação; §5 a §9 especificam a implementação; §10 a §12 organizam a execução.

Sumário

- 0. Veredito e nota
 - 1. O que a v3.0 acerta (e por que isso importa)
 - 2. Os 18 achados, por severidade
 - 3. Correções críticas em nível de especificação
 - 3.1 C1 — O commit point não existe
 - 3.2 C2 — Webhook PAYMENT_SETTLED não autenticado
 - 3.3 C3 — /demo/clock é bypass total de expiração
 - 3.4 A1 — Vazamento de orçamento no recibo
 - 3.5 A2 — Correlação e rastreamento
 - 3.6 A3 — Griefing de reserva
 - 3.7 A4 — Replay da requisição assinada
 - 3.8 A5 — Política de indisponibilidade
 - 3.9 A6 — Rotação de chaves e JWKS
 - 3.10 M1-M8 — Achados médios
 - 4. A camada de revogação em tempo real
 - 4.1 Compatibilidade com AP2 v0.2
 - 4.2 Onde colocar a referência de revogação
 - 4.3 Commit-reveal vs. chave de revogação
 - 4.4 Construção do Revocation Registry
 - 4.5 Race conditions e o commit point formal
 - 4.6 ACTIVE → COMMITTED com chamada externa
 - 4.7 Retries e idempotência
 - 4.8 Fail-closed: a política correta
 - 4.9 Cache e latência de revogação
 - 4.10 As cinco superfícies de replay
 - 4.11 Privacidade
 - 4.12 Authorization proof one-time
 - 4.13 Múltiplos processors concorrentes
 - 4.14 O que reutilizar em vez de inventar
 - 5. Arquitetura de implementação: visão geral
 - 6. Modelo de dados completo
 - 7. Máquinas de estado e caminhos críticos
 - 8. Superfície de API v4
 - 9. MVA vs. production-grade
 - 10. Testes, SLOs e observabilidade
 - 11. Plano de execução hora a hora
 - 12. Demo v4, riscos e planos B
- A. Novos reason codes
- B. Novos vetores de conformidade
- C. Menu do juiz v4
- D. Checklist contra o enunciado
- E. Fontes e revalidação

0. Veredito e nota

A v3.0 é um documento incomum. A grande maioria dos times de hackathon entrega um agente mais esperto; este documento entrega um *motor de autoridade* com invariantes nomeados, vetores de conformidade, testes de propriedade e um menu de ataques impresso para o juiz. A tese está certa — o problema do enunciado é de autoridade, não de pagamento — e a arquitetura sustenta a tese. Os defeitos que restam são de duas naturezas: **o documento para onde o dinheiro começa a se mover**, e **tudo que só aparece em produção** (privacidade, disponibilidade, latência, rotação de chaves).

0.1 Rubrica

Critério	Peso	Nota	Justificativa
Aderência ao enunciado (requisitos + bônus)	25%	10,0	Cobre os cinco requisitos obrigatórios e os três bônus. O §26.2 é um mapeamento linha a linha. Nada do enunciado ficou sem endereço.
Correção técnica do núcleo (política, concorrência, evidência)	20%	9,0	Motor de duas fases, contadores sob lock, cadeias por subject, revalidação no capture. Perde ponto por não fechar a fronteira transacional do pagamento (C1).
Modelo de ameaças e defesa	15%	8,5	27 ataques mapeados com defesa e resultado esperado. Faltam três superfícies reais: webhook forjado, griefing de reserva e replay da requisição assinada.
Demonstrabilidade e trial by fire	15%	10,0	Console do juiz, relógio controlável, agente adversarial, reset, quatro abas ao vivo. É a melhor parte do documento e é exatamente o que o enunciado exige.
Prontidão de produção (money movement, disponibilidade, latência)	10%	5,0	Nenhum orçamento de latência, nenhuma política de degradação, nenhum tratamento de resultado desconhecido do PSP. Para um componente <i>inline</i> no caminho do pagamento, isso é a lacuna mais cara.
Privacidade e conformidade (LGPD, PCI)	5%	3,0	Praticamente ausente. O recibo entregue ao merchant vaza o orçamento mensal do humano; o ledger append-only colide com direito à eliminação; a saída de escopo PCI não é declarada — sendo que é um argumento comercial forte.
Realismo de execução (escopo × tempo)	5%	5,0	Doze blocos, quatro superfícies de UI, um LLM, testes de propriedade e um agente adversarial. O roadmap é excelente como ordem de prioridade e irrealista como plano.
Posicionamento e narrativa de produto	5%	8,5	A leitura da trajetória pública da Yuno é precisa e a frase de venda funciona. Falta o modelo econômico: quem paga, quanto, e qual a transferência de responsabilidade contratual.

NOTA FINAL DO DOCUMENTO V3.0: 8,5 / 10

Nota projetada do projeto entregue, se executado exatamente como o roadmap descreve: **7,5** — não por qualidade, mas por risco de escopo: doze blocos em um fim de semana produzem, com alta probabilidade, uma cena principal frágil.

Teto alcançável com as correções deste briefing e o plano de execução do §11: **9,5**. O 0,5 restante é o que nenhum hackathon entrega — PSP real, certificação ACP e PKI de verdade.

0.2 As três frases que resumem a avaliação

- A v3.0 protege a autorização perfeitamente e o pagamento não protege ninguém.** O documento termina no COMMIT; do capture e o dinheiro começa depois dele. Entre o commit local e a resposta da rede existe uma janela onde o sistema não tem estado definido — e é justamente a janela em que um juiz curioso vai desligar a rede.
- O elo mais fraco da cadeia de entrega é um webhook sem assinatura declarada.** A v3.0 corrigiu, com razão, a regra "entregue em PAYMENT_SETTLED, não em VALID" — mas se esse evento é apenas um POST HTTP, forjá-lo é o ataque mais barato do sistema inteiro e entrega mercadoria de graça sem tocar em nenhuma assinatura.
- O sistema conhece tudo que o humano compra e não diz o que faz com isso.** Uma camada que se instala antes de todo pagamento agentic é o observador mais bem posicionado do mercado. Declarar essa propriedade e desenhar contra ela — pseudônimos pareados, divulgação seletiva, privacidade de rebanho na consulta de status — é diferencial de produto, não capricho acadêmico.

1. O que a v3.0 acerta, e por que isso importa

Registrado aqui para que nenhuma dessas decisões seja revertida durante a implementação. Elas são o produto.

Decisão	Por que está certa
Invariante I2 — todo campo avaliado vem assinado por quem tem autoridade sobre ele	É a única defesa estrutural contra prompt injection que existe. Filtros de palavra são jogo de gato e rato; um motor cujo tipo de entrada não contém nenhum campo de origem do agente não pode ser injetado, por construção. Mantenha a separação no nível do <i>tipo</i> , não da convenção.
Invariante I6 — a chave do humano nunca existe do lado de quem a avalia	Sem isso, toda a matriz de disputa é decorativa. É a primeira pergunta de um juiz que trabalha com pagamentos, e o botão "tentar assinar pelo servidor → 403" é a melhor resposta de dez segundos do documento inteiro.
Invariante I3 — ALLOW no passado não significa ALLOW agora	Dois pontos de decisão com revalidação completa no segundo. É o que transforma "revogação" de slide em propriedade observável, e é a cena que ganha o trial by fire.
Invariante I4 — reserva transacionada sob lock	Fatiamiento concorrente é o ataque que o juiz executa com dois cliques e que a maioria dos times não defende. Somar committed + reserved em vez de só committed é a diferença entre passar e falhar ao vivo.
Motor de duas fases (gates duros / avaliadores suaves)	Elimina a dependência de ordem que produziu o defeito D1 e devolve <i>todos</i> os motivos de falha ao auditor. A propriedade P7 (embaralhar a fase B não muda decisão) é um teste barato com valor de prova.
Separação terms_hash / offer_hash	É o que faz o caminho de escalação fechar. Vincular aprovação humana aos <i>termos</i> e não à <i>emissão</i> é sutil, correto, e resolve reemissão, hold e prorrogação de uma vez.
Banda de três faixas (allow / escalate / ceiling)	Modela a frase do enunciado sem torção e ainda resolve um problema humano real: existem valores que a pessoa não quer nem ser tentada a aprovar às três da manhã.
Cadeias de ledger por subject + âncoras de época	Resolve a bifurcação da cadeia única sem lock global. Tem, além disso, uma propriedade que a v3.0 não explora e deveria (ver §3.10-M6): <i>eventos que podem correr entre si estão sempre na mesma cadeia</i> , então a ordem relativa entre revogação e decisão é provável por construção.
Vetores de conformidade + replay determinístico	Transforma "entregamos uma aplicação" em "entregamos um protocolo com implementação de referência". É o argumento de posicionamento mais forte que um hackathon pode produzir.
Console do juiz e menu impresso	Muda a dinâmica da avaliação de "onde isso quebra" para "o que quero ver funcionar". Vale mais do que qualquer slide de arquitetura.

2. Os 18 achados, por severidade

Críticos quebram no palco ou tornam o produto inseguro em produção. Altos são explorações reais que um avaliador técnico encontra. Médios são lacunas que um juiz sênior nota e um cliente exigiria.

ID	Sev.	Achado	Impacto em uma linha	Seção
C1	CRÍT	O commit point não existe: a chamada ao PSP não participa da transação local e não há máquina de estado para o resultado	Timeout do PSP deixa o sistema sem estado definido: ou cobra sem registrar, ou registra sem cobrar, ou queima orçamento do humano em um pagamento que não aconteceu.	3.1
C2	CRÍT	Webhook PAYMENT_SETTLED sem autenticação especificada	A regra "entregue só em PAYMENT_SETTLED" vira o ataque mais barato do sistema: um POST forjado faz o merchant entregar mercadoria de graça sem tocar em nenhuma assinatura.	3.2
C3	CRÍT	POST /demo/clock é um endpoint god-mode que anula toda expiração	Se vazara para o build de produção, qualquer chamador desativa validade de oferta, de recibo, de token, de mandato e de janela cumulativa com uma requisição.	3.3
A1	ALTO	counters_snapshot no AuthorizationReceipt entregue ao merchant	A VuelaYa passa a saber o teto mensal da Marta, quanto ela já gastou e quantos usos restam. Informação de precificação, vazada por desenho.	3.4
A2	ALTO	mandate_id visível ao merchant e status_url derivada dele	Dois merchants correlacionam o mesmo humano; o registry acumula o grafo completo de compras de cada usuário. É exatamente o risco que a pesquisa de revogação levanta na pergunta 11.	3.5
A3	ALTO	Griefing de reserva: nenhum limite de reservas vivas por mandato	Um agente com bug ou hostil mantém 100% do teto reservado indefinidamente. Zero dinheiro se move e o humano fica sem poder comprar. Ataque ausente da tabela §17.	3.6
A4	ALTO	Assinatura da requisição do agente sem janela anti-replay própria	A v3.0 protege nonce de oferta e nonce de decisão, mas não o nonce da requisição. Uma requisição capturada pode ser reenviada dentro da validade da oferta.	3.7
A5	ALTO	Nenhuma política declarada de indisponibilidade de dependência	"Fail closed" aparece como corolário filosófico, não como matriz operável. Pior: fail-closed na autoridade não pode significar liberar a reserva — isso é fail-open no contador.	3.8
A6	ALTO	Sem rotação de chaves nem JWKS para verificação offline	O merchant verifica o recibo offline e o arquiva como evidência por 540 dias. Se a camada rotacionar a chave, a evidência de disputa deixa de verificar.	3.9
M1	MÉD	Nenhum orçamento de latência para um componente inline	O gate entra no caminho crítico do pagamento. Merchants medem approval rate e abandono em milissegundos. O documento não cita latência uma única vez.	3.10
M2	MÉD	Ledger append-only x LGPD; escopo PCI não declarado	Direito à eliminação colide com imutabilidade. E a saída de escopo PCI — nenhum PAN em nenhum artefato — é um argumento comercial que o documento não usa.	3.10
M3	MÉD	NUMERIC(12,2) assume duas casas decimais	Quebra para JPY e CLP (zero casas) e para KWD, BHD e JOD (três). Um erro de fator 100 no teto de gastos.	3.10
M4	MÉD	Semântica de atributos não restringidos não é declarada	Mandato para "voo para COR" não diz nada sobre classe, bagagem ou reembolsabilidade — e o motor aprova em silêncio o que não foi restringido.	3.10
M5	MÉD	POST /offers/{id}/hold exigido do merchant sem fallback	Impõe integração nova a cada merchant para que a escalação funcione. Existe um caminho equivalente que não exige nada dele.	3.10
M6	MÉD	Multi-nó e múltiplos processores concorrentes não endereçados	Réplicas de leitura servindo status introduzem lag de replicação; dois gates independentes sobre o mesmo mandato são um problema de gasto duplo.	3.10
M7	MÉD	Fonte de tempo confiável e skew entre merchant e camada	A oferta carrega expires_at do relógio do merchant e o gate compara com o seu. Alguns segundos de skew produzem OFFER_EXPIRED falso ou aceitam oferta vencida.	3.10
M8	MÉD	Sem métricas, SLOs ou tela de prova de desempenho	"Não existe race" e "é rápido" são afirmações. Um painel de p99 ao vivo e 10.000 sequências aleatórias são evidências.	3.10
E1	EXEC	Escopo de doze blocos sem plano temporal nem divisão de papéis	É o maior risco à nota final. O roadmap ordena por risco corretamente, mas não diz quem faz o quê nem em quantas horas.	11

O CORTE DE LEITURA, SE VOCÊ TIVER QUINZE MINUTOS

Implemente **C1** (máquina de estado do pagamento com outbox), **C2** (webhook assinado + confirmação por pull) e **A1** (recibo com duas visões). Essas três mudanças custam cerca de meio dia somado e são as únicas que alteram o *desenho* do sistema; todo o resto são acréscimos que encaixam sem refatoração.

3. Correções críticas em nível de especificação

Cada correção segue o mesmo formato: o que a v3.0 diz, por que está errado ou incompleto, e a especificação substituta — em detalhe suficiente para ser implementada sem novas decisões de desenho.

3.1 C1 — O commit point não existe

O que a v3.0 diz

O §15 apresenta o SQL do capture: abre transação, trava o mandato, roda C1-C10, marca `consumed_at`, move os contadores de `reserved` para `committed`, grava `captures` e `ledger`, e commita. O §5 diz que o adaptador de pagamento "emite token escopado consume-once e encaminha a intenção de pagamento" *somente após ALLOW revalidado*. Em nenhum ponto o documento diz **onde**, em relação a esse `COMMIT`; , a chamada externa acontece.

Por que está errado

Só existem três posições possíveis e todas as três estão quebradas sem estado adicional:

- **Dentro da transação.** A linha do mandato fica travada durante uma chamada de rede de 200-2000 ms. Todo o argumento de concorrência do §12 evapora: o teste de "vinte requisições simultâneas" passa a medir a latência do PSP, e um PSP lento vira um lock timeout em cascata. Pior: se o PSP cobra e a transação local dá rollback por qualquer motivo, existe cobrança sem decisão registrada — o cenário exato que a matriz de disputa do §19 não consegue julgar, porque nenhuma evidência foi persistida.
- **Depois do commit.** Os contadores já foram movidos para `committed`. Se o PSP recusar, o teto mensal da Marta foi consumido por um pagamento que não existiu, e não há caminho de compensação especificado. O invariante I4 ("nenhum efeito econômico sem contador transacionado") passa a valer ao contrário: existe contador sem efeito econômico.
- **Sem resposta (timeout).** Não há estado para isso. Se a camada tratar timeout como falha e liberar a reserva, e a transação tiver sido de fato autorizada na rede, o resultado é gasto duplo — a liberação da reserva é *fail-open* no contador. Se tratar como sucesso, um erro de rede vira cobrança fantasma no extrato do humano.

A FORMULAÇÃO CORRETA DO COMMIT POINT

O commit point **não** é a resposta da rede de pagamento. É a transição de estado local que torna a submissão inevitável. Formalmente:

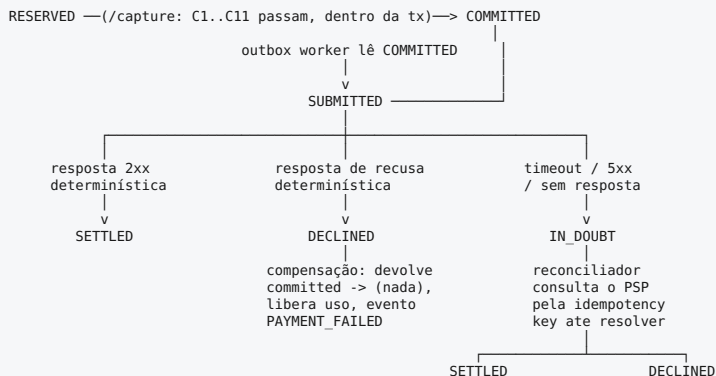
O commit point é a inserção da linha no outbox, dentro da mesma transação que consome a decisão. Antes dela, a autoridade é revogável. Depois dela, a revogação continua válida para usos futuros do mandato e deixa de afetar esta tentativa específica, que passa a ser tratada pelos mecanismos normais de pagamento — reversal, cancelamento, estorno, disputa.

Isso é exatamente a propriedade que a pesquisa de revogação pede, e ela é alcançável sem transação distribuída, sem 2PC e sem coordenação com a Mastercard.

A especificação substituta

Introduz-se a entidade `payment_attempt`, com máquina de estado explícita, e um **outbox transacional**. A regra de ouro: *nenhuma chamada externa dentro de transação de banco; nenhum "sem resposta" tratado como falha; nenhuma reserva liberada sem resposta negativa definitiva ou reconciliação que prove ausência de cobrança*.

ESTADOS DE `payment_attempt`



REGRA INVIOUVEL: `IN_DOUBT` nunca libera reserva nem contador.
O orcamento permanece consumido ate que exista resposta definitiva.
Fail-closed na autoridade; fail-conservador no contador.

Por que outbox e não chamada direta. O padrão transaccional outbox resolve o problema de "duas escritas, um commit" sem transação distribuída: a decisão de pagar e a intenção de pagar são gravadas atomicamente no mesmo banco; um worker separado lê a intenção e executa a chamada externa com *at-least-once delivery*. A idempotência do lado do PSP transforma *at-least-once* em *effectively-once*. A chave de idempotência enviada ao PSP é o `decision_id` — que já é único, já é consume-once e já está no recibo assinado. Não é preciso inventar identificador novo.

```
-- CAPTURE v4: a transacao que contem o commit point
BEGIN;
SELECT * FROM mandates WHERE id = $m FOR UPDATE;
SELECT * FROM mandate_counters
  WHERE mandate_id = $m AND window_key = $d.window_key FOR UPDATE;

-- C1..C10 do $15 da v3.0, integrais, sem atalho

UPDATE decisions SET consumed_at = now()
  WHERE id = $d AND consumed_at IS NULL;          -- C11: 0 linhas = replay

UPDATE mandate_counters
  SET reserved      = reserved - $a,
      committed     = committed + $a,
      uses_reserved  = uses_reserved - 1,
      uses_committed = uses_committed + 1
  WHERE mandate_id = $m AND window_key = $d.window_key;

INSERT INTO payment_attempts (id, decision_id, state, amount_minor, currency,
                             psp_idempotency_key, created_at)
  VALUES ($p, $d, 'COMMITTED', $a_minor, $ccy, $d /* = decision_id */, now());

INSERT INTO outbox (id, topic, payload, available_at, attempts) -- <<< COMMIT POINT
  VALUES ($o, 'payment.submit', jsonb_build_object('attempt_id', $p), now(), 0);

INSERT INTO captures (agent_id, idempotency_key, response_body, status_code, ...);
INSERT INTO ledger (...);          -- CAPTURE_COMMITTED
COMMIT;

-- A partir daqui a linha do mandato esta LIVRE. Nenhum lock atravessa a rede.
```

```
-- OUTBOX WORKER (processo separado, N replicas, concorrencia segura)

loop:
BEGIN;
SELECT * FROM outbox
  WHERE topic='payment.submit' AND available_at <= now()
  ORDER BY available_at
  FOR UPDATE SKIP LOCKED LIMIT 1;          -- sem contencao entre workers
UPDATE payment_attempts SET state='SUBMITTED', submitted_at=now()
  WHERE id=$p AND state IN ('COMMITTED', 'SUBMITTED');
COMMIT;

resp := psp.authorize(token, idempotency_key = attempt.psp_idempotency_key)
  with timeout = 8s

match resp:
  2xx definitivo -> state=SETTLED; ledger PAYMENT_SETTLED; notifica merchant
  recusa definitiva -> state=DECLINED; compensar(); ledger PAYMENT_FAILED
  timeout | 5xx | erro de rede ->
    state=IN_DOUBT
    available_at = now() + backoff(attempts)  -- 1s,2s,4s,8s,30s,2min,10min
    NUNCA compensar aqui

-- compensar(): devolve committed -> 0 para esta tentativa,
-- uses_committed -= 1, grava evento COMPENSATION_APPLIED.
-- Roda em transacao propria, sob o mesmo lock do mandato.
```

```
-- RECONCILIADOR (roda a cada 60s e no boot)

para cada payment attempt em IN_DOUBT ou SUBMITTED ha mais de 120s:
  r := psp.get_by_idempotency_key(attempt.psp_idempotency_key)
  if r.found and r.approved: state = SETTLED
  if r.found and r.declined: state = DECLINED; compensar()
  if not r.found and idade > janela_de_autorizacao_do_psp:
    state = DECLINED; compensar()
  else:
    permanece IN_DOUBT (orcamento segue consumido)

-- Alarme operacional: qualquer attempt IN_DOUBT ha mais de 15 min e incidente.
-- 0 humano ve na tela: "pagamento em confirmacao" - nunca "aprovado".
```

O que muda nas telas e no ledger

- O recibo do humano ganha um terceiro estado visível: **autorizado, em confirmação, liquidado**. Um sistema honesto mostra a incerteza em vez de escondê-la.
- Novos eventos de ledger: `CAPTURE_COMMITTED`, `PAYMENT_SUBMITTED`, `PAYMENT_IN_DOUBT`, `PAYMENT_RECONCILED`, `COMPENSATION_APPLIED`.
- Novos reason codes: `PAYMENT_DECLINED`, `PAYMENT_IN_DOUBT`, `COMPENSATION_APPLIED` (Anexo A).
- Nova evidência de disputa **E8**: tentativa de pagamento com estado final e chave de idempotência, provando que a decisão gerou no máximo uma submissão à rede.

A CENA DE DEMO QUE ISSO DESBLOQUEIA — E VALE MAIS QUE A REVOGAÇÃO

No console do juiz, um botão **"derrubar o PSP"**. O juiz inicia uma compra e desliga o processador durante o capture. A tela do humano mostra *"pagamento em confirmação"*, o contador continua reservado, e nenhuma mercadoria é entregue. O juiz religa o PSP; o reconciliador resolve em segundos e a tela vira *liquidado*. **Nenhum outro time de hackathon vai demonstrar isso**, porque a maioria nem terá percebido que o problema existe — e é a pergunta que qualquer engenheiro de pagamentos faz nos primeiros três minutos.

3.2 C2 — O webhook PAYMENT_SETTLED não está autenticado

O problema

A v3.0 corrigiu corretamente a regra de entrega: o merchant verifica em `VALID` e entrega em `PAYMENT_SETTLED` (§8.4, passo 5). Mas o passo 5 depende de um webhook, e o documento não especifica assinatura, timestamp, nonce, política de retry nem verificação do lado do merchant. Um webhook sem autenticação transforma toda a arquitetura de assinaturas em decoração: o atacante não precisa forjar mandato, oferta, recibo ou token — basta um POST para o endpoint do merchant dizendo que o pagamento liquidou.

É o ataque mais barato do sistema inteiro e ele não está na tabela §17.

A correção, em três camadas

1. **O webhook é apenas uma dica.** Reformular a semântica: `PAYMENT_SETTLED` notifica; ele não autoriza entrega. A fonte de verdade é o *pull*: `GET /decisions/{handle}/status` retornando `PAYMENT_SETTLED`. O merchant que perder o webhook faz polling com backoff e chega ao mesmo resultado. Isso remove o webhook do caminho de confiança e ainda resolve entrega perdida.
2. **Assinatura do envelope.** Todo webhook carrega `Signature-Input` e `Signature` no formato RFC 9421, sobre `@method`, `@target-uri`, `content-digest`, `created` e `nonce`, com a chave da camada resolvível via JWS (§3.9). O merchant valida a assinatura, rejeita `created` fora de ± 300 s e mantém cache de nonces vistos pela mesma janela.
3. **Idempotência e ordem no lado do merchant.** O evento carrega `event_id` e `sequence` por decisão; reentregas são detectadas por `event_id`; eventos fora de ordem são descartados por `sequence` menor que o último aplicado. Política de retry: 8 tentativas com backoff exponencial até 6 h, depois fila morta com alarme.

```
WEBHOOK v4 (o merchant NUNCA entrega so com isto)

POST https://merchant.example/hooks/authz
Content-Digest: sha-256=:X48E9q0okqrvdts8n0JRJN3OWDUoyWxBf7kbu9DBPE=:
Signature-Input: sig1=("@method" "@target-uri" "content-digest" "created" "nonce");
               keyid="key_authz_02"; created=1787059800; nonce="9f2c...41"
Signature: sig1=:MEUCIQ...

{ "event_id": "evt_7c1...", "sequence": 3, "type": "PAYMENT_SETTLED",
  "decision_handle": "dh_8b31...", // handle opaco, NAO o decision_id
  "terms_hash": "sha256:...", "settled_at": "2026-08-29T14:05:02Z" }

REGRAS DE ENTREGA DO MERCHANT (substitui o passo 5 do §8.4):
1. valida assinatura do webhook -> se falhar, ignora e nao alarma o time
2. GET /decisions/{decision_handle}/status -> exige PAYMENT_SETTLED (fonte de verdade)
3. so entao libera a mercadoria; grava DELIVERY_RELEASED como evidencia E4b
```

Impacto na matriz de disputa: nova linha em §19.2 — *merchant entregou com base apenas em webhook, sem confirmação por pull* → disputa procedente, responsável o merchant. E novo ataque na tabela §17 e no menu do juiz: **"forjar liquidação"**, botão que dispara um POST não assinado para o merchant; resultado esperado: mercadoria não é liberada e o evento aparece no ledger como `WEBHOOK_SIGNATURE_INVALID`.

3.3 C3 — /demo/clock é um bypass total de expiração

O problema

A v3.0 comemora, com razão, que `now` é parâmetro do motor e que por isso o relógio acelerável "é de graça". É de graça para o demo e catastrófico para produção: `POST /demo/clock` desativa, em uma requisição, validade de oferta, validade de recibo, validade de token, `valid_until` do mandato, TTL de escalação e virada de janela cumulativa. É um único endpoint que anula seis controles de segurança simultaneamente. Endpoints de demo vazam para produção com frequência histórica alta.

A correção

- **Separação em tempo de compilação, não de configuração.** As rotas `/demo/*` vivem em um pacote `@authz/demo-harness` que só é importado quando `BUILD_PROFILE=demo`. No perfil `prod` o pacote não entra no bundle — o código não existe, e portanto não pode ser habilitado por variável de ambiente esquecida.
- **Asserção de boot.** O serviço recusa iniciar se `BUILD_PROFILE=prod` e qualquer rota registrada casar com `/demo`, ou se `NODE_ENV=production` com o harness carregado. Falha ruidosa, no boot, antes de aceitar tráfego.
- **Dois fontes de tempo, tipadas.** `WallClock` (produção, monotônico + NTP) e `LogicalClock` (demo, com offset). O motor recebe `now`: `Instant` e nunca lê relógio próprio; o adaptador é injetado no boot. O tipo do offset não existe no perfil `prod`.
- **Evento de ledger para cada salto.** `CLOCK_ADVANCED { from, to, actor }`. Um auditor que veja a linha do tempo do demo consegue distinguir o que expirou naturalmente do que foi acelerado. Isso também protege o time: se um juiz perguntar "vocês não estão trapaceando com o relógio?", a resposta está no ledger.

VALE DIZER ISSO NO PALCO

"O relógio controlável existe para o senhor conseguir ver expiração em cinco segundos. Ele não existe no build de produção — o pacote inteiro não é compilado — e todo salto que dermos aqui aparece no ledger como evento assinado." Custa oito segundos e responde antecipadamente à suspeita mais natural sobre uma demo com controle de tempo.

3.4 A1 — O recibo entregue ao merchant vaza o orçamento do humano

O problema

O `AuthorizationReceipt` do §8.4 contém `counters_snapshot` com `per_tx_allow_max`, `cumulative_limit`, `cumulative_committed`, `cumulative_reserved`, `uses_committed` e `max_uses`. Esse objeto é entregue ao merchant, que o verifica offline e o arquiva. Consequências:

- A `VuelaYa` passa a saber que a Marta tem US\$ 600 por mês, gastou US\$ 130 e pode comprar mais duas vezes. Isso é insumo direto de precificação discriminatória — e, num mundo de compradores agêntic, saber o teto do comprador é a informação comercialmente relevante.
- A camada estaria distribuindo dados financeiros do usuário a terceiros sem base legal clara sob a LGPD, e sem que o humano veja isso em lugar nenhum da interface.
- É desnecessário: o merchant precisa saber que a compra *foi* autorizada, não *quanto sobrou*.

A correção: divulgação seletiva com raiz assinada

O recibo passa a existir em duas visões derivadas do mesmo objeto assinado. Em vez de assinar o JSON inteiro, a camada assina uma **raiz de divulgação**: cada campo é comprometido individualmente com um salt, e a assinatura cobre a raiz Merkle desses compromissos. Quem recebe uma visão parcial verifica a assinatura da raiz e os caminhos Merkle dos campos que recebeu, sem nunca ver os campos ocultos — e sem que a camada precise assinar duas vezes ou manter dois objetos divergentes.

```

CONSTRUCAO (uma assinatura, N visoes)

para cada campo i do recibo:
    leaf_i = SHA256( JCS({ path_i : value_i }) || salt_i )
disclosure_root = MerkleRoot( leaf_0 .. leaf_n ) // ordem canonica por path
signature       = Ed25519( JCS({ disclosure_root, decision_id, decided_at,
                                expires_at, authorizer_key_id }) )

VISA0 DO MERCHANT (o que ele precisa para entregar e para se defender)
decision, decision_handle, offer_id, offer_hash, terms_hash, merchant_id,
category, amount, decided_at, expires_at, mandate_version_binding (opaco),
policy_trace_hash, disclosure_root, provas Merkle dos campos acima, signature

VISA0 DO PRINCIPAL / AUDITOR (tudo)
+ mandate_id, principal_id, agent_id, counters_snapshot, window_key,
reason_codes, trace completo, salts

PROPRIEDADE: o merchant verifica UMA assinatura e sabe que os campos que
recebeu pertencem ao mesmo objeto que o auditor vera integralmente.
Nenhuma das partes precisa confiar na outra sobre o conteudo oculto.

```

Custo de implementação: cerca de 60 linhas (construção da árvore, geração de prova, verificação). É o tipo de detalhe que um juiz técnico reconhece imediatamente como "essas pessoas já pensaram em privacidade", e resolve um problema real em vez de decorativo. Se o tempo apertar, o fallback aceitável é **emitir dois objetos assinados separadamente** (`receipt.merchant` e `receipt.principal`), ambos contendo `decision_id` e `terms_hash` — pior criptograficamente, mas resolve o vazamento em quinze minutos.

3.5 A2 — Correlação e rastreamento entre merchants e pelo registry

O problema

Três vazamentos de identificador, todos por desenho:

- `mandate_id` e `principal_id` aparecem no recibo entregue ao merchant. Dois merchants que troquem dados — ou um adquirente que atenda ambos — reconstróem o histórico de compras do mesmo humano entre lojas.
- `status_url` é derivada do `decision_id` e o `decision_id` é vinculável ao mandato. Cada consulta de status informa ao operador da camada exatamente quem está comprando o quê, e quando.
- Um `revocation_id` ou `revocation_commitment` fixo no mandato — como propõe a pesquisa — é um identificador estável e público que qualquer verificador correlaciona ao longo do tempo.

A correção, em quatro medidas

Medida	Como
Pseudônimo pareado por (mandato, merchant)	<code>pairwise_id = HMAC(k_layer, mandate_id merchant_id)</code> , truncado a 128 bits. O merchant recebe <code>pairwise_id</code> , nunca <code>mandate_id</code> . Dois merchants comparando bases não encontram nenhum identificador em comum. É o mesmo mecanismo dos <i>pairwise subject identifiers</i> do OpenID Connect — padrão maduro, zero invenção.
Handle opaco por decisão	<code>decision_handle</code> = 128 bits aleatórios, sem derivação do <code>decision_id</code> , com índice próprio. É o único identificador que sai da camada em direção ao merchant e ao webhook. O <code>decision_id</code> permanece interno e no ledger.
Privacidade de rebanho na consulta de status	A verificação offline usa uma <i>status list</i> no formato W3C Bitstring Status List: uma lista comprimida cobrindo dezenas de milhares de posições. Quem quer saber se um mandato está revogado baixa a lista inteira e olha o bit localmente — o servidor não aprende <i>qual</i> mandato foi consultado. Ver §4.4 e §4.11.
Minimização no ledger	Texto livre do agente, título do item e destino não entram no ledger em claro: entram como <code>sha256(salt valor)</code> , com o valor no cofre de PII cifrado por chave de <code>subject</code> (§3.10-M2). O auditor vê o hash e, com autorização do humano, o valor.

O LIMITE HONESTO, QUE DEVE SER DECLARADO NO DOCUMENTO E DITO NO PALCO

Uma camada de autorização instalada antes de todo pagamento agentic é, por construção, o observador mais bem posicionado do mercado: ela vê cada compra de cada usuário. Nenhuma engenharia elimina isso. O que se pode fazer é (a) minimizar o que é retido, (b) impedir que *terceiros* correlacionem, e (c) posicionar a camada do lado do usuário — como infraestrutura de carteira, análoga a um credential provider AP2 — e não do lado do merchant. Declarar esse limite em §26.3 vale mais do que omiti-lo e ser perguntado.

3.6 A3 — Griefing de reserva: negação de orçamento sem mover dinheiro

A tabela de ameaças §17 cobre gasto indevido, mas não cobre **impedimento de gasto legítimo**. O ataque: um agente com bug — ou hostil, ou simplesmente um segundo agente enrolado sob o mesmo principal — chama `/authorize` repetidamente e nunca captura. Cada ALLOW reserva orçamento por 90 segundos. Com um pedido a cada 80 segundos, 100% do teto mensal permanece reservado indefinidamente. Nenhum centavo se move, nenhum reason code dispara, e a Marta simplesmente não consegue mais comprar. O reaper de 5 s não ajuda: ele libera reservas vencidas, e o atacante emite novas.

Correção

- **Teto de reservas vivas por mandato:** `limits.max_live_reservations`, padrão **1**, configurável até 3. Uma segunda tentativa enquanto há reserva viva recebe `DENY RESERVATION LIMIT` — não é fila, é recusa, e o agente correto reage capturando ou desistindo.
- **Orçamento de reserva desperdiçada:** contador rolante de reservas expiradas sem capture por agente. Acima de N (padrão 5) em 10 minutos, o agente entra em *cooldown* exponencial e o humano recebe uma notificação: "seu agente está reservando orçamento e não concluindo compras".
- **Rate limit por (agent_id, mandate_id)** no `/authorize`, com `Retry-After`, e por IP no ingress.
- **Reserva menor que a janela de escalção:** quando o resultado é `ESCALATE`, a reserva vive o TTL da escalção, não 90 s — o que é correto, mas significa que um agente pode congelar orçamento por 120 s escalando de propósito. O teto de reservas vivas resolve isso também.

Novo ataque na tabela §17 e no menu do juiz: **"prender o orçamento"** — botão que dispara dez autorizações sem capture; resultado esperado: a primeira reserva, as nove seguintes recebem `RESERVATION LIMIT`, e o painel do humano mostra o alerta.

3.7 A4 — Replay da requisição assinada do agente

A v3.0 tem anti-replay para **oferta** (`offer_sightings`), para **decisão** (`uq_decision_nonce`, `consumed_at`) e para **capture** (idempotency key). Não tem para a **requisição HTTP assinada**. A assinatura estilo RFC 9421 é descrita como "subconjunto dos headers" sem dizer quais, e sem janela nem cache de nonce. Consequência: uma requisição `/authorize` capturada em rede pode ser reenviada; ela falhará por `OFFER_NONCE_REPLAY` por sorte, não por desenho — e uma requisição `/capture` ou `/escalations/{id}/resolve` capturada não tem essa proteção acidental.

PERFIL DE ASSINATURA OBRIGATORIO (RFC 9421, subconjunto declarado)

Signature-Input: sig1=("method" "@target-uri" "content-digest"
"authorization-agent" "created" "expires" "nonce");
keyid="key_agt_7f2"; alg="ed25519"

REGRAS DE VERIFICACAO (todas fail-closed):

1. created <= now + 2s (skew tolerado, nunca a favor do cliente)
2. expires - created <= 60s (janela curta e obrigatória)
3. now < expires
4. nonce nunca visto para este agent_id -> tabela request_nonces, TTL 120s
5. content-digest confere com o corpo recebido byte a byte
6. @target-uri e o host real, nao o do proxy (impede replay cross-endpoint)

Novos reason codes: REQUEST_SIGNATURE_STALE, REQUEST_NONCE_REPLAY,
REQUEST_DIGEST_MISMATCH

O cache de nonces vive em tabela Postgres com índice único e TTL varrido pelo reaper — Redis é otimização, não requisito. Custo: uma tabela, um índice único, dez linhas de verificação.

3.8 A5 — Política de indisponibilidade: a matriz que falta

A v3.0 declara "falhar fechado é sempre defensável" como corolário e um kill switch global. Isso é filosofia correta e operação insuficiente: um operador de plantão precisa saber, por dependência, o que o sistema faz. E há uma sutileza que a formulação genérica esconde e que é o erro conceitual mais comum nesse tipo de sistema:

FAIL-CLOSED NA AUTORIDADE ≠ FAIL-CLOSED NO CONTADOR

Negar uma autorização quando não se consegue verificar a revogação é fail-closed correto: o pior caso é uma venda perdida.

Liberar uma reserva quando não se consegue verificar o resultado do pagamento **parece** conservador e é o oposto: libera orçamento que talvez já tenha sido gasto, permitindo gasto duplo. O correto é **manter o orçamento consumido até prova em contrário**.

Regra: na dúvida, negue autoridade e preserve consumo.

Dependência	Caminho crítico	Comportamento na indisponibilidade	Justificativa e degradação
Postgres primário (mandatos, contadores)	Sim	503 em /authorize e /capture. Nenhuma decisão emitida.	Sem contador transacionado não há autorização possível. Failover síncrono para réplica standby; RTO alvo 30 s.
Estado de revogação	Sim	DENY REVOCATION_UNVERIFIABLE.	É a propriedade central do produto. Autorizar sem poder verificar revogação é vender uma garantia que não se tem.
Ledger	Sim (por desenho)	Transação inteira falha; nenhuma decisão sem evento.	Custo de disponibilidade deliberado: o ledger é escrito na mesma transação. Declarar isso é melhor do que descobrir que decisões existem sem trilha.
Merchant (/offers/{id}/hold)	Não	Escalação segue sem hold; usa-se equivalência de reemissão (§3.10-M5).	Hold é otimização. A aprovação vincula-se a terms_hash, que é estável.
PSP / adaptador Yuno	Sim, pós-commit	Attempt vai a IN_DOUBT; retry com backoff; contador permanece consumido.	Ver §3.1. Nunca compensar por ausência de resposta.
Fila de outbox	Não	Nada acontece até o worker voltar; decisões continuam sendo tomadas.	Outbox no próprio Postgres significa que a fila cai junto com o banco e não antes dele — uma dependência a menos.
SSE / notificações	Não	Telas caem para polling de 3 s.	Nenhuma decisão depende de push. A escalação expira por TTL de qualquer forma.
Serviço de escalação (console humano)	Não	Escalações pendentes resolvem por ESCALATION_TIMEOUT → DENY.	on_timeout: DENY já é o padrão. Indisponibilidade do console vira recusa, não aprovação.
JWKS público	Não	Merchants usam chave em cache; TTL 24 h com sobreposição de 30 dias entre chaves.	Verificação offline não pode depender de disponibilidade da camada.

Escada de degradação, em ordem: (1) modo normal; (2) *read-heavy degradado* — status e ledger continuam servindo, autorizações negam; (3) *halt* — kill switch global, DENY_SYSTEM_HALTED em tudo, capture inclusive; (4) *drain* — nenhuma decisão nova, apenas reconciliação de attempts pendentes. Os quatro estados são um único flag no banco, visíveis em todas as telas, e o console do juiz tem um botão para cada.

3.9 A6 — Rotação de chaves e verificação offline duradoura

O §7.3 diz que merchant e camada usam "chaves estáticas em variável de ambiente, sem rotação" e declara PKI fora de escopo. Como decisão de hackathon, correto. Como lacuna de especificação, não: o §8.4 pede que o merchant verifique o recibo **offline** e o §19 pede que ele o archive como evidência de disputa — e a janela de chargeback de cartão chega a 540 dias. Uma chave rotacionada em seis meses invalida a defesa do merchant exatamente quando ela é necessária.

Especificação mínima (implementável em uma hora, mesmo no hackathon)

- **Endpoint JWKS** em GET /.well-known/jwks.json, com todas as chaves — ativas e aposentadas — cada uma com kid, alg: EdDSA, crv: Ed25519, use: sig, valid_from, valid_until e status: ACTIVE | RETIRED | COMPROMISED.
- **Regra de verificação histórica:** um recibo verifica se a chave que o assinou estava ACTIVE em decided_at — não em *agora*. Chave RETIRED continua verificando o passado; chave COMPROMISED invalida assinaturas posteriores a compromised_at e preserva as anteriores, exatamente como o §7.2 já faz com a chave do agente. A simetria é elegante e é a resposta pronta se o juiz perguntar.
- **Sobreposição de 30 dias** entre chave nova e antiga; publicação da nova 24 h antes do uso, para que caches de merchant já a tenham.
- **Assinatura do próprio JWKS** por uma chave raiz offline, cujo fingerprint é distribuído fora de banda no onboarding do merchant. Sem isso, comprometer o servidor de JWKS é comprometer tudo.
- **Rotação da chave do merchant** é evento de ledger (MERCHANT_KEY_ROTATED) e ofertas assinadas com a chave antiga continuam válidas até expirarem — validade de 120 s torna isso trivial.

3.10 Achados médios: M1 a M8

M1 — Orçamento de latência para um componente inline

O documento posiciona a camada "imediatamente antes da execução do pagamento" e não menciona latência uma única vez. Para o cliente-alvo — merchants e infraestrutura de pagamento — latência adicionada no caminho de autorização é a métrica que decide adoção, porque ela se converte em abandono e em queda de approval rate. Definir o orçamento é barato e é diferencial de credibilidade.

Etap	p50 alvo	p99 alvo	Nota
decide() puro, em memória	0,2 ms	2 ms	Função pura, sem I/O. Medida por benchmark, não estimada.
P0ST /authorize ponta a ponta	18 ms	60 ms	Dominado por 1 RTT de banco com lock de linha + escrita de ledger na mesma transação.
P0ST /capture ponta a ponta	22 ms	80 ms	Revalidação completa + outbox. Não inclui a chamada ao PSP, que é assíncrona por desenho (§3.1).
GET /decisions/{h}/status	4 ms	15 ms	Leitura indexada, no -store, sempre no primário.
Overhead total no fluxo de pagamento	~40 ms	~140 ms	Meta declarada: abaixo de 150 ms no p99 . Acima disso, a camada precisa justificar sua existência comercialmente.

Análise de contenção, que é a pergunta seguinte: o SELECT ... FOR UPDATE serializa por mandato. Com transação de ~15 ms, o teto é ~60 autorizações por segundo *por mandato* — irrelevante para um humano, relevante para um mandato corporativo de reposição. Se necessário, a saída é

particionar contadores por *bucket* dentro do mandato (N linhas somando o mesmo teto, escolha aleatória de bucket, com verificação exata sob lock de todas as linhas apenas quando o saldo restante fica abaixo de um limiar). Declare a análise; implemente só se sobrar tempo. **Não** o mencione como feito.

M2 — LGPD versus ledger imutável, e a saída de escopo PCI

Dois pontos que um juiz sênior ou um cliente corporativo levanta imediatamente:

- **Direito à eliminação × append-only.** Um ledger encadeado por hash não pode ter linhas apagadas sem quebrar a cadeia — que é justamente a propriedade demonstrada. A conciliação padrão é *crypto-shredding*: o ledger armazena apenas pseudônimos e hashes; os dados pessoais (nome, e-mail, título do item, destino do voo) vivem em um cofre cifrado com uma chave por titular. Eliminar = destruir a chave. Os hashes permanecem, a cadeia continua íntegra, e o dado se torna irrecuperável. Retenção declarada: 540 dias para evidência de disputa, alinhada à janela de chargeback, com base legal de exercício regular de direitos.
- **Escopo PCI DSS.** A v3.0 diz, corretamente, que nenhum PAN existe em nenhum artefato — e depois não extrai o valor disso. Deveria: uma camada que manipula apenas *referências* de cofre e network tokens permanece fora do escopo de PCI DSS, o que significa que um merchant pode adotá-la sem alterar sua avaliação de conformidade. Para quem vende para times de pagamento, essa é uma das três primeiras perguntas. Uma linha em \$26 e um selo na tela do merchant.

M3 — Unidades monetárias

`NUMERIC(12,2)` assume duas casas decimais universais. JPY e CLP não têm casas decimais; KWD, BHD, JOD e TND têm três. Um teto de "1000" em KWD interpretado com duas casas é um erro de fator dez. Correção: armazenar `amount_minor BIGINT + currency CHAR(3) + exponent SMALLINT` derivado de ISO 4217, com formatação apenas na borda de apresentação. Comparações só entre a mesma moeda — o gate A10 já garante isso. Nenhuma conversão cambial dentro de um mandato, nunca: um mandato é monomoeda e essa restrição é uma *feature*, porque câmbio dentro de limite de gasto é uma superfície de ataque por variação de taxa.

M4 — Semântica de atributos não restringidos

O escopo do mandato é uma *allowlist sobre as chaves declaradas*: um mandato que restringe `destination` não diz nada sobre classe tarifária, bagagem, reembolsabilidade ou número de conexões. O motor aprova em silêncio tudo que não foi restringido — comportamento correto, mas não declarado, e é uma pergunta natural do juiz ("e se ele comprar primeira classe?"). Correções:

- Declarar a semântica explicitamente na especificação e na tela de criação do mandato ("atributos não listados não são avaliados").
- Acrescentar `scope.required_attributes: [...]` — chaves que devem *existir* na oferta, sob pena de `ATTRIBUTE_MISSING`.
- Acrescentar `scope.attribute_policy: "lenient" | "strict"`. Em `strict`, qualquer chave presente na oferta e ausente do conjunto conhecido resulta em **ESCALATE** com `UNDECLARED_ATTRIBUTE`. É três linhas de código, é uma condição rica genuinamente nova, e é uma boa resposta de palco.

M5 — Fallback quando o merchant não implementa hold

Exigir `POST /offers/{id}/hold` impõe integração nova a cada merchant só para que a escalacão funcione — atrito de adoção significativo para um produto cuja tese é "o merchant adota porque ganha defesa em disputa". Mas a v3.0 já resolveu isso sem perceber: a aprovação humana vincula-se ao `terms_hash`, que é **estável através de reemissões**. Logo:

EQUIVALÊNCIA DE REEMIÇÃO

Qualquer oferta nova cujo `terms_hash` seja idêntico ao aprovado satisfaz a aprovação, tenha ela vindo de um `hold` explícito ou de uma reemissão espontânea do merchant. **Hold é otimização, não requisito.** O merchant que não o implementa continua funcionando: o agente simplesmente pede uma oferta nova com os mesmos termos após a aprovação. Declarar isso reduz a integração obrigatória do merchant a dois endpoints — catálogo e emissão de oferta — e transforma uma objeção comercial em nota de rodapé.

M6 — Multi-nó, réplicas e múltiplos processors

- **Múltiplos nós de API:** seguro sem alteração. O `SELECT ... FOR UPDATE` serializa entre processos, não dentro de um. Requisito: todas as escritas e **todas as leituras de autoridade** vão ao primário. Uma leitura de status servida por réplica com 200 ms de lag pode reportar `ACTIVE` para um mandato já revogado — o pior bug possível neste produto. Marcar as consultas de autoridade com hint de roteamento obrigatório para o primário e ter um teste que falha se alguma delas for para réplica.
- **Ordem entre revogação e decisão:** vale explicitar uma propriedade que a v3.0 tem de graça e não usa — *revogação e decisão do mesmo mandato estão sempre na mesma cadeia de ledger*, porque `chain_key = mandate_id` e ambas passam pelo mesmo lock. Portanto a ordem relativa entre elas é totalmente ordenada e provável, sem depender das âncoras de época. Isso responde diretamente ao juiz que perguntar "e se eu revogar exatamente no mesmo instante?". Não existe "mesmo instante": existe uma ordem, e o ledger a exhibe.
- **Múltiplos processors concorrentes** sobre o mesmo mandato: ver §4.13. Resumo: é o problema do gasto duplo e não tem solução sem um sequenciador único ou consenso. A resposta correta é que o mandato nomeia sua autoridade de autorização, e o token consume-once é o recurso exclusivo.

M7 — Fonte de tempo e skew

A oferta carrega `expires_at` gerado pelo relógio do merchant; o gate compara com o seu. Regras: (a) o gate é a única autoridade de tempo para decisão; (b) tolerância de skew de **±2 s, sempre contra o cliente** — nunca estende validade, apenas encurta; (c) oferta com `issued_at` no futuro além da tolerância é rejeitada com `OFFER_CLOCK_SKEW`; (d) intervalos internos (TTL, reaper) usam relógio monotônico, não de parede, para sobreviver a ajustes de NTP; (e) o timestamp do ledger é o do gate, e a divergência com o do merchant é registrada como contexto — em disputa, ela importa.

M8 — Observabilidade e a tela de prova

Instrumentação mínima, e uma decisão de encenação: transformar métricas em evidência visível. Métricas: `authz_decisions_total{outcome,reason}`, `authz_latency_seconds{route}` (histograma), `counters_lock_wait_seconds`, `payment_attempts_by_state`, `outbox_lag_seconds`, `in_doubt_attempts`, `ledger_chain_verify_seconds`, `revocation_propagation_seconds`. A aba do juiz ganha um rodapé permanente com quatro números ao vivo: **p99 de /authorize, attempts em dúvida, divergências de replay e gasto autorizado fora do mandato: US\$ 0,00**. Custa uma hora e converte três afirmações do pitch em leitura direta.

4. A camada de revogação em tempo real

Esta seção responde às catorze perguntas da pesquisa anexa e integra o resultado à especificação da camada. A avaliação é feita **sem assumir que a proposta está correta**, como pedido. Adiantando o veredito: a tese central — *um gate de autorização imediatamente antes da submissão à rede, com commit point formal* — está certa e preenche uma lacuna declarada do AP2. Dois dos mecanismos propostos, porém, devem ser substituídos: o commit-reveal como mecanismo primário de revogação, e a operação `authorize_and_commit` como primitiva atômica que atravessa a rede.

4.1 Pergunta 1 — Compatibilidade com a arquitetura do AP2 v0.2

Compatível, e mais do que isso: complementar por lacuna declarada.

O AP2 estrutura autoridade em credenciais verificáveis assinadas pelo usuário — *Intent Mandate* (condições sob as quais o agente pode comprar quando o humano não está presente), *Cart/Checkout Mandate* (o carrinho específico) e *Payment Mandate* (compartilhado com rede e emissor, sinalizando envolvimento de agente e presença humana). Cada um existe em estágio *open* (restrições para execução autônoma) e *closed* (autorização de um checkout finalizado).

O ACHADO QUE SUSTENTA O POSICIONAMENTO INTEIRO

O AP2 é uma especificação jovem — v0.2, de abril de 2026 — e a revogação de mandatos exige tratamento cuidadoso: mandatos criptográficos são não-repudiáveis por desenho, o que é a característica de segurança, mas não existe mecanismo no protocolo para que o usuário revogue uma autorização já emitida. Se o usuário assina um Intent Mandate autorizando gastos ao longo de um mês e muda de ideia no terceiro dia, a superfície atual do protocolo depende do credential provider ou da carteira para aplicar a revogação fora do AP2.

Ou seja: **a camada proposta não conflita com o AP2 — ela é exatamente o componente que o AP2 delega para fora de si e não especifica.** Isso deve ser dito em uma frase no pitch e uma no documento. Muda a leitura de "mais um protocolo proprietário" para "a peça que falta no protocolo que a indústria está adotando".

Três implicações práticas de compatibilidade:

- **Não desassinar nada.** A premissa da pesquisa está correta: o mandato AP2 permanece criptograficamente válido; o que se acrescenta é *estado vivo* consultável. Um mandato assinado é uma afirmação sobre o passado ("o usuário autorizou"); revogação é uma afirmação sobre o presente ("essa autoridade ainda vale"). Confundir as duas é o erro conceitual que a proposta evita corretamente.
- **Onde a camada se encaixa no modelo de papéis.** No vocabulário AP2, o gate ocupa a posição de *credential provider* combinada com verificação delegada do *merchant payment processor* — que é precisamente onde o AP2 diz que a revogação deve morar. Não é um papel novo no protocolo, é um papel existente que ninguém implementou.
- **Identidade de agente vem de fora do AP2.** O AP2 vincula o mandato à identidade do *usuário* pela chave de assinatura, não à do agente; a atestação de identidade do agente é esperada de uma camada como o Visa Trusted Agent Protocol. A v3.0 já resolve isso com a cerimônia de enrollment do §7.2 — mantenha, e cite a relação com o TAP: o enrollment assinado é a versão mínima da mesma propriedade.

4.2 Pergunta 2 — Onde exatamente incluir a referência de revogação

Dentro do corpo assinado do mandato, no campo `credentialStatus` do modelo de credenciais verificáveis do W3C — não em um campo proprietário.

Como o mandato AP2 é uma VC, ele herda `credentialStatus` de graça. Usar o campo padrão significa que qualquer verificador de VC já sabe o que fazer com ele, e que a camada não inventa protocolo onde já existe um.

```
MANDATE (corpo assinado) — extensao de revogacao v4

{
  "id": "urn:uuid:...",
  "type": ["VerifiableCredential", "IntentMandate"],
  "issuer": "did:key:z6Mk...", // o HUMANO
  "credentialSubject": { ...escopo, limites, condicoes... },

  "credentialStatus": [
    { "id": "https://status.authz.example/lists/2026-09#41217",
      "type": "BitstringStatusListEntry",
      "statusPurpose": "revocation",
      "statusListIndex": "41217", // posicao ALEATORIA na lista
      "statusListCredential": "https://status.authz.example/lists/2026-09" },
    { "id": "https://status.authz.example/lists/2026-09-susp#41217",
      "type": "BitstringStatusListEntry",
      "statusPurpose": "suspension", // suspender != revogar
      "statusListIndex": "41217",
      "statusListCredential": "https://status.authz.example/lists/2026-09-susp" }
  ],

  "authorizationAuthority": { // extensao MINIMA nossa
    "endpoint": "https://gate.authz.example",
    "authority_id": "authz_br_1",
    "max_status_staleness_seconds": 10, // contrato de frescor
    "requires_realtime_check": true // hot path NAO usa a lista
  },

  "panicCommitment": "sha256:9f3c...", // OPCIONAL, ver §4.3
  "proof": { ... } // assinatura Ed25519 do humano
}
```

Três decisões dentro desse desenho, cada uma com motivo:

- **statusListIndex é aleatório, não sequencial nem derivado do mandate_id.** Índice sequencial revela ordem de emissão e permite enumerar; índice derivado do identificador do mandato reintroduz correlação. Sorteia-se uma posição livre na lista.
- **Duas listas, dois propósitos.** `revocation` é terminal; `suspension` é reversível. A v3.0 já tem os dois estados no mandato (§8.2) e o padrão W3C já os distingue — alinhar é gratuito.
- **requires_realtime_check: true é o que impede o mau uso.** A lista de status serve para verificação *interoperável e offline* por terceiros (um segundo processor, um auditor, um emissor). O caminho quente da autorização **nunca** a consulta: ele lê o estado no primário, sob lock, na mesma transação. Sem essa distinção explícita, alguém implementa "chechar a lista" no `/capture` e reintroduz a janela de propagação que a arquitetura toda existe para eliminar.

O que não fazer: criar `revocation_id`, `revocation_authority` e `revocation_registry` como três campos proprietários soltos, como o rascunho da pesquisa considera. São três nomes novos para o que `credentialStatus` já expressa, com a desvantagem de que nenhum verificador existente os entende — e a desvantagem adicional de que um `revocation_id` estável é um identificador de rastreamento perfeito (§3.5).

4.3 Pergunta 3 — Commit-reveal oferece vantagem real sobre assinar a revogação?

Como mecanismo primário: não. Como capacidade delegável de emergência: sim, e é a única coisa que ele faz que a assinatura não faz.

A proposta é: o mandato carrega $C = \text{SHA256}(R)$; revogar é publicar R ; enquanto R não aparecer no registry, a autorização vale. A alternativa convencional é: o humano assina uma chamada de revogação com a chave que ele já usa para assinar o mandato.

Dimensão	Commit-reveal (SHA256(R))	Revogação assinada (chave Ed25519 já existente)
Autenticação de quem revogou	Nenhuma. Prova apenas conhecimento de R . O registry não sabe se foi o titular, a carteira ou quem interceptou o segredo.	Prova de posse da chave do principal. A disputa tem um autor identificado, com timestamp.
Granularidade	Um bit, uma vez. Não expressa suspender, retomar, baixar limite de 600 para 200, restringir categoria ou revogar só a chave do agente.	Qualquer operação, porque o payload assinado descreve a transição. A v3.0 já precisa disso: §9.2 tem cinco operações distintas de redução de poder.
Rotação e recuperação	R é irrotacionável: mudar R exige reemitir e reassinar o mandato. Perder R é perder a capacidade de revogar.	Rotação de chave é operação normal, com evento de ledger; perda de dispositivo já tem caminho definido (re-enrollment revoga tudo).
Anti-replay	Não aplicável — mas também não necessário, porque publicar R duas vezes é idempotente.	Nonce + versão de destino no payload assinado, como §9.2 já exige. Custa uma linha.
Evidência em disputa	Prova que alguém publicou R em T . Não prova <i>quem</i> . Em disputa "eu revoguei antes", isso é fraco.	Prova assinada de quem revogou e quando. É evidência E9 na matriz do §19.
Custódia	Um segredo a mais para guardar, com o mesmo problema de custódia da chave privada e nenhuma das ferramentas maduras (WebAuthn, HSM, passkey).	Zero segredos adicionais. Reaproveita a chave não-extraível do §7.1.
Não-revogação é provável?	Não. "R nunca foi publicado" é uma <i>prova de ausência</i> : depende de o verificador ter visão completa e honesta do registry. Um registry que censura esconde a revogação e ninguém percebe.	Sim. Uma <i>status assertion</i> assinada com <code>validFrom/validUntil</code> é uma afirmação positiva, verificável offline, e um registry que mente produz evidência do que mentiu.
Privacidade	Nenhuma vantagem: C está no mandato em claro, é estável e é um identificador de correlação perfeito. Depois da revelação, R é público.	Índice aleatório em bitstring com privacidade de rebanho (§4.11) é estritamente melhor.
Delegabilidade sem conceder autoridade	Vantagem real e exclusiva. Entregar R a um terceiro dá a ele o poder de <i>desligar</i> sem dar o poder de <i>gastar</i> .	Exige emitir uma credencial ou token separado com escopo de revogação — mais trabalho, mas também mais controlável.

RECOMENDAÇÃO

Revogação assinada como mecanismo primário. Ela reaproveita a chave que já existe, cobre as cinco operações de redução de poder que a v3.0 já precisa, produz evidência de autoria e permite afirmação positiva de estado.

Commit-reveal preservado apenas como "cápsula de pânico" — `panicCommitment` no mandato. O humano pode imprimir R em papel, guardar num cofre, entregar ao sócio, à equipe de operações ou a um *dead-man switch*. Quem tiver R desliga o mandato e não pode fazer mais nada com ele. Isso é exatamente o "link de pânico com token de uso único" que a v3.0 já lista em §9.2, agora com forma criptográfica adequada e verificável por terceiros.

Note a assimetria que justifica os dois mecanismos coexistirem: **revogar é a operação que precisa ser a mais fácil do sistema**. Se ela exigir o dispositivo com a chave, e o dispositivo for o problema, o desenho falhou. A cápsula de pânico é a resposta para esse caso — e apenas para ele.

4.4 Pergunta 4 — Como construir o Revocation Registry

Arquitetura	Latência de revogação	Privacidade	Resolve gasto duplo?	Avaliação
Serviço centralizado HA	ms	Ruim	Sim	Único que entrega consistência forte no caminho quente. O operador vê tudo — mitigável com pseudônimos e minimização, não eliminável. Escolha para MVA e para produção v1.
Hospedado pelo emissor (carteira do usuário)	ms-s	Boa	Não	Alinhado ao AP2 (o credential provider é quem revoga). Problema fatal no caminho quente: a disponibilidade da carteira de cada usuário vira dependência do pagamento. Use como origem autoritativa da intenção , replicada para o gate, nunca como consulta síncrona.
Distribuído com consenso (Raft/quórum)	ms-dezenas de ms	Ruim	Sim	O que produção de verdade exige quando há múltiplas regiões. Um líder por partição de mandato. Evolução natural da v1 , mesma semântica, custo operacional maior.
Blockchain / smart contract	s-min	Péssima	Sim	Desqualificado para o caminho quente. Finalidade em segundos ou minutos dentro de um fluxo de autorização de cartão que precisa fechar em ~150 ms é inviável, e o custo de gas por verificação é absurdo. Útil apenas como <i>sumidouro de âncoras</i> : publicar periodicamente a raiz Merkle do ledger para obter irrefutabilidade contra o próprio operador — que é exatamente o limite declarado em §26.3 da v3.0.
W3C Status List / acumulador	segundos (cadência de publicação)	Ótima	Não	Melhor privacidade disponível (rebanho) e melhor interoperabilidade. Não serve sozinho ao caminho quente por causa da defasagem de publicação. Camada de verificação para terceiros.

ARQUITETURA RECOMENDADA: HÍBRIDA EM TRÊS CAMADAS

- 1. Autoridade de tempo real** — Postgres primário (v1) ou grupo Raft (v2), consultado sob lock na mesma transação da decisão. Latência de revogação efetiva: **zero**, porque não há propagação: quem decide e quem revoga tocam a mesma linha.
- 2. Status list publicada** — bitstring assinada, republicada a cada 10 s, com `validFrom/validUntil`, para verificação offline por merchants, auditores e outros processors. Contrato de frescor declarado no mandato (`max_status_staleness_seconds`).
- 3. Âncoras Merkle** — raiz das cabeças de cadeia a cada época, opcionalmente publicada fora, para irrefutabilidade contra o operador.

A camada 1 dá **corretude**; a 2 dá **interoperabilidade e privacidade**; a 3 dá **auditabilidade**. Nenhuma delas sozinha entrega as três.

4.5 Pergunta 5 — Como evitar race conditions entre revogação e execução

A pesquisa identifica corretamente o problema — `T0` gate consulta `ACTIVE`, `T1` usuário revoga, `T2` gate submete — e conclui, também corretamente, que um `isRevoked()` por HTTP não basta. A conclusão seguinte, porém, precisa de ajuste: a solução **não** é uma primitiva `authorize_and_commit` que abrange a submissão à rede. É separar o problema em dois, um dos quais é trivial e o outro é resolvido por `outbox`.

O erro a evitar: check-then-act distribuído

Consultar estado em um serviço e agir em outro é uma *time-of-check to time-of-use* clássica. Qualquer desenho que consulte o registry por rede e depois submeta tem a janela; encurtá-la (cache curto, consulta mais próxima da submissão) reduz a probabilidade e não elimina a condição. Sistemas de pagamento não podem ter propriedades probabilísticas nesse ponto.

A formulação correta

REGRA 1 – Verificacao e efeito na MESMA transacao, sobre a MESMA linha.

```
BEGIN;
  SELECT status, version FROM mandates WHERE id=$m FOR UPDATE;  -- (*)
  ...C1..C11...
  UPDATE decisions SET consumed_at = now() WHERE ...;
  INSERT INTO payment_attempts (state='COMMITTED') ...;
  INSERT INTO outbox ...;                                     -- COMMIT POINT
COMMIT;
```

REGRA 2 – A REVOGACAO pega o MESMO lock.

```
BEGIN;
  SELECT ... FROM mandates WHERE id=$m FOR UPDATE;  -- mesma linha de (*)
  UPDATE mandates SET status='REVOKED';
  -- libera reservas vivas no window_key de cada uma
  -- invalida escalacoes pendentes
  -- marca decisoes vivas como RELEASED
  INSERT INTO ledger (MANDATE_REVOKED);
COMMIT;
```

CONSEQUENCIA: as duas transacoes sao TOTALMENTE ORDENADAS pelo lock. Nao existe "ao mesmo tempo". Uma delas ocorre primeiro e a outra ve o resultado. O ledger – cadeia unica por mandato – exibe a ordem, e ela e a mesma para o humano, para o merchant, para o auditor e para o juiz.

A RESPOSTA DE DEZ SEGUNDOS PARA O JUIZ QUE REVOGAR "NO MESMO INSTANTE"

"Não existe mesmo instante. As duas operações disputam o mesmo lock de linha, então uma acontece antes da outra — e o ledger mostra qual. Se a revogação ganhou, o capture morre com `MANDATE_REVOKED`. Se o capture ganhou, o pagamento já estava comprometido e a revogação vale para os próximos usos. Em nenhum dos dois casos o resultado é ambíguo, e em nenhum dos dois o dinheiro sai fora do que a senhora autorizou."

Por que `authorize_and_commit(mandate_id, transaction_hash)` não deve abranger a rede

A intuição da pesquisa está certa — é preciso uma transição atômica `ACTIVE → COMMITTED` versus `ACTIVE → REVOKED`, com apenas uma vencedora. Mas se essa operação incluir a chamada à Mastercard, ela deixa de ser atômica: uma chamada de rede não tem resultado binário, tem três (sucesso, falha, *desconhecido*), e a terceira não cabe numa transição de dois estados. A correção é deslocar a fronteira: **a transição atômica é local e termina no outbox; a submissão à rede é uma consequência inevitável, não uma parte da transição**. Ver §4.6.

4.6 Pergunta 6 — `ACTIVE → COMMITTED` com uma chamada externa não-transacional

Esta é a pergunta mais importante da pesquisa e a resposta está desenvolvida em §3.1. Resumindo o desenho e acrescentando a justificativa formal:

PROPRIEDADE DESEJADA

"Um usuário pode retirar a autoridade até o instante em que uma transação específica e irrevogavelmente aceita para submissão à rede."

PROVA INFORMAL (com o desenho de outbox)

1. A inserção no outbox ocorre dentro da transação `T_c` que trava o mandato.
 2. A revogação ocorre dentro de uma transação `T_r` que trava o mesmo mandato.
 3. Locks de linha impõem ordem total entre `T_c` e `T_r`.
 4. Se `T_r < T_c`: `T_c` le `status=REVOKED` e falha em `C4`. Nada vai ao outbox. Logo, nada é submetido. [revogação venceu]
 5. Se `T_c < T_r`: a linha de outbox existe e é durável. O worker submeterá, com `at-least-once` + `idempotencia` = `effectively-once`. `T_r` vale para usos futuros do mandato. [compra venceu]
 6. Não existe estado em que a linha de outbox exista e a submissão não venha a ocorrer, exceto perda total do banco — que também perderia a decisão, mantendo consistência.
- => 0 commit point e a inserção no outbox. QED (informal).

Três consequências que precisam estar no documento:

- **Depois do commit point, revogação não é mais o instrumento.** Os instrumentos passam a ser os do mundo de pagamentos: *reversal*, cancelamento, estorno e disputa. Isso é exatamente o que a pesquisa propõe e está correto. Vale explicitar na tela do humano: enquanto o estado é *autorizado*, o botão diz "cancelar"; depois de *liquidado*, ele diz "contestar". A mudança de rótulo comunica a fronteira sem uma linha de explicação.
- **O `transaction_hash` da pesquisa tem um papel real, mas outro.** Ele não é o argumento de uma primitiva atômica; ele é o **vínculo do commit**: `COMMITTED(transaction_hash)` registra *qual* transação foi comprometida, o que impede que um retry vire compra nova e que uma segunda transação reaproveite o mesmo commit. Na v3.0 esse papel já é desempenhado por `decision_id + terms_hash`; formalizar `transaction_hash = SHA256(decision_id || terms_hash || amount_minor || currency)` e usá-lo como chave de idempotência no PSP unifica os dois vocabulários.
- **Nenhuma parte disso exige cooperação da Mastercard.** A propriedade é obtida inteiramente do lado de cá, com um banco relacional. Esse é um argumento de viabilidade forte: o produto não depende de mudança em nenhuma rede.

4.7 Pergunta 7 — Retries e idempotência

Há **três** níveis de idempotência e a pesquisa trata de um. Todos os três precisam existir, com chaves diferentes e donos diferentes:

Nível	Chave	Dono	Regra
Agente → camada	(agent_id, Idempotency-Key)	Agente	Resposta original persistida e devolvida com Idempotent-Replay: true, mesmo status code. Mesma chave com corpo diferente → 409 IDEMPOTENCY_KEY_REUSE. Já está na v3.0 e está correto.
Decisão → pagamento	transaction_hash (derivado de decision_id)	Camada	Uma decisão gera no máximo uma payment_attempt; índice único em payment_attempts(decision_id). Um retry do worker reenvia a mesma chave ao PSP.
Camada → PSP	psp_idempotency_key = transaction_hash	PSP	At-least-once do outbox vira effectively-once porque o PSP deduplica. Se o PSP não suportar idempotência, o reconciliador consulta por referência antes de reenviar — e nunca reenvia cegamente.

A regra que impede retry de virar compra nova já está garantida por consumed_at (C11): uma decisão consumida não pode ser consumida de novo, com nenhuma chave de idempotência. A chave de idempotência controla a resposta; o consumed_at controla o efeito. São controles distintos e é importante que a implementação não confunda um com o outro — confundi-los é como se transforma um retry legítimo de rede em erro 500, que é pior que o problema original.

4.8 Pergunta 8 — Política correta se o serviço de revogação estiver indisponível

A hipótese da pesquisa está certa: fail-closed. Com duas qualificações que mudam a implementação, já desenvolvidas em §3.8:

- Fail-closed vale para a autoridade, não para o contador.** Negar autorização por não conseguir verificar revogação: correto. Liberar reserva por não conseguir verificar o pagamento: fail-open disfarçado. Na dúvida, negue autoridade e preserve consumo.
- A assimetria de risco justifica a rigidez.** O pior caso de um falso negativo é uma venda perdida e um cliente irritado; o pior caso de um falso positivo é uma cobrança não autorizada, um chargeback e a destruição do argumento de venda inteiro. Não há simetria, então não há razão para modo degradado permissivo — e "o sistema negou algo que talvez pudesse permitir" é uma história que se conta em dez segundos.

Um refinamento que vale mencionar e não implementar: em cenários de indisponibilidade prolongada, uma política de allow-list de emergência (transações abaixo de um valor mínimo, para merchants de baixo risco) é o que sistemas de cartão fazem com stand-in processing. É defensável em produção com um comitê de risco por trás. Em um hackathon, mencione que conhece o mecanismo e que deliberadamente não o implementou.

4.9 Pergunta 9 — Cache e latência de revogação aceitável

Superfície	Cache permitido	Regra
Decisão de autorizar (/authorize)	Zero	Leitura no primário, sob lock, na mesma transação. Não há cache porque não há consulta remota.
Revalidação no capture	Zero	Idem. É o ponto onde o dinheiro se move.
Status consultado pelo merchant	≤ 5 s	Cache-Control: no-store permanece para a resposta; o merchant pode manter valor local por até 5 s para exibição. Entrega exige leitura fresca (§3.2).
Status list publicada	≤ 10 s	validFrom/validUntil assinados. Verificador que aceitar lista mais velha que max_status staleness_seconds do mandato está fora de conformidade — e o vetor de conformidade testa isso.
JWKS	24 h	Chaves mudam com semanas de antecedência; sobreposição de 30 dias.

SLO de revogação, declarado e medido: p99 do intervalo entre o clique de revogar e o primeiro DENY consequente é inferior a 1 segundo — dominado inteiramente pela latência de uma transação de banco e pelo round-trip da interface. A métrica revocation_propagation_seconds existe para provar isso e aparece no rodapé da aba do juiz.

4.10 Pergunta 10 — As cinco superfícies de replay

A v3.0 cobre três; a v4 cobre cinco. Enumerá-las separadamente evita a armadilha de achar que "temos anti-replay" é uma propriedade única.

#	Superfície	Controle	Reason code
1	Requisição HTTP assinada do agente	nonce + created/expires ≤ 60 s + cache de nonces NOVO §3.7	REQUEST_NONCE_REPLAY, REQUEST_SIGNATURE_STALE
2	Oferta assinada do merchant	offer_sightings(merchant_id, nonce) único + expires_at	OFFER_NONCE_REPLAY, OFFER_EXPIRED
3	Decisão / recibo / token	consumed_at com UPDATE condicional + índice único parcial em decisões vivas	TOKEN_ALREADY_CONSUMED, OFFER_ALREADY_AUTHORIZED
4	Aprovação de escalção	Vínculo a (terms_hash, mandate_version) + max_uses: 1 + TTL	APPROVAL_INVALID
5	Webhook de liquidação	Assinatura + created ±300 s + cache de nonce no merchant + confirmação por pull NOVO §3.2	WEBHOOK_SIGNATURE_INVALID, WEBHOOK_REPLAY

Uma sexta, mencionada para completude: **replay de corpo assinado em PATCH**, já resolvido na v3.0 pela versão de destino dentro do payload assinado (VERSION_CONFLICT). Mantém-se.

4.11 Pergunta 11 — Implicações de privacidade do registry

A pergunta identifica o risco central corretamente: um registry consultado a cada compra aprende o grafo completo de compras de cada usuário. Quatro mecanismos, em ordem de custo:

- Privacidade de rebanho (custo: baixo, ganho: alto).** A status list é uma bitstring comprimida cobrindo 131.072 posições. Quem quer saber se um mandato está revogado baixa a lista inteira — algumas dezenas de kilobytes comprimidos — e lê o bit localmente. O servidor não aprende qual posição interessa. É o mecanismo padrão do W3C Bitstring Status List, existe pronto e é a resposta mais forte disponível sem criptografia exótica.
- Pseudônimos pareados (custo: baixo, ganho: alto).** §3.5. Nenhum identificador comum entre merchants.
- Minimização do que o gate retém (custo: baixo, ganho: médio).** O gate precisa de categoria, valor, moeda, merchant e atributos restringidos. Não precisa do título do item, da descrição, do nome do passageiro nem do texto do agente — tudo isso entra como hash, com valor no cofre cifrado por titular.
- Recuperação privada de informação, PIR (custo: alto, ganho: alto).** Permite consultar um bit sem revelar qual. Mencione como direção; não implemente. Rebanho já entrega 95% do benefício por 2% do custo.

O QUE NÃO PODE SER RESOLVIDO E DEVE SER DECLARADO

O gate em tempo real vê cada transação — é o que lhe permite decidir. A mitigação honesta não é criptográfica, é **de posicionamento**: a camada é infraestrutura *do lado do usuário*, análoga a um credential provider AP2 ou a uma carteira, com dever fiduciário e retenção declarada — e não um observador do lado do merchant vendendo inteligência de mercado. Escreva isso em §26.3. Um juiz que pense em privacidade vai valorizar a franqueza mais do que uma afirmação de que o problema não existe.

4.12 Pergunta 12 — Emitir um authorization proof curto e one-time?

Sim — e a v3.0 já tem o objeto certo com o nome errado. O `ScopedPaymentToken` (§8.5) é precisamente esse *proof*: 60 s, consume-once, vinculado a uma decisão, a um merchant e a um valor. O que falta é dizer **o que ele prova** e reconhecer **o que ele custa**.

```
ScopedPaymentToken v4 = AUTHORIZATION PROOF one-time

{
  "token_id"       : "tok...",           // UNIQUE, consume-once
  "transaction_hash" : "sha256:...",     // vinculo do commit
  "decision_handle" : "dh...",          // opaco
  "authority_id"    : "authz_br 1",
  "authority_asserted_at": "2026-08-29T14:04:41Z",
  "authority_valid_until": "2026-08-29T14:05:41Z", // 60s
  "assertion": "AUTHORITY_LIVE_AT_ASSERTION_TIME", // afirmacao POSITIVA
  "payment_ref": "pm_ref_visa_4821",         // referencia de cofre, nunca PAN
  "amount_minor": 13000, "currency": "USD", "exponent": 2,
  "merchant_id": "mch_vuelaya",
  "signature": "..."                      // chave da camada, verificavel via JWKS
}
```

Por que isso é melhor do que depender da consulta ao registry:

- O processador e a rede verificam **offline**, com uma chave pública. O registry deixa de ser um ponto único de falha e um gargalo de rastreamento no caminho de todos os pagamentos.
- É uma afirmação positiva e assinada ("a autoridade estava viva às 14:04:41"), não uma prova de ausência. Em disputa, isso é evidência E9.
- Encaixa no modelo AP2 sem invenção: é o tipo de metadado estruturado que o *Payment Mandate* carrega para dar visibilidade a PSP, rede e emissor sobre envolvimento de agente.

O CUSTO, QUE PRECISA ESTAR ESCRITO

O TTL do token é, exatamente, a janela cega de revogação. Dentro desses 60 segundos, uma revogação não impede aquela submissão específica — e isso é uma consequência lógica, não um bug: é a definição do commit point. Duas regras decorrem daí:

- O TTL deve ser **o menor possível** — o suficiente para submeter, não para o agente pensar. 60 s é generoso; 15 s é defensável se o adaptador do PSP for local.
- O token só é emitido **depois** do commit point, nunca antes. Emitir antes recria a race que toda a arquitetura existe para eliminar.

4.13 Pergunta 13 — Múltiplos processadores/verifiers concorrentes sobre o mesmo mandato

É o problema do gasto duplo, e vale ser direto sobre isso: **não tem solução sem um recurso exclusivo compartilhado**. Dois gates independentes, cada um consultando uma status list e cada um decidindo por conta própria, autorizam ambos o mesmo mandato contra o mesmo saldo. Nenhuma quantidade de criptografia resolve — é um problema de consenso, e as únicas saídas conhecidas são um sequenciador único ou um protocolo de consenso.

Modelo	Análise
Autoridade nomeada (recomendado)	O mandato declara <code>authorizationAuthority.authority_id</code> . Qualquer processador que não seja a autoridade não decide : ele chama, recebe um <code>ScopedPaymentToken</code> consume-once e submete. Como a consumação é centralizada, dois processors não podem consumir o mesmo token. Simples, correto, e é o que a v3.0 já faz sem ter nomeado.
Lease exclusivo	Variante para quando o processador precisa de autonomia temporária: a autoridade concede um lease sobre uma fatia do orçamento (ex.: US\$ 200 por 5 min), o processador decide localmente dentro dele, e devolve o saldo não usado. Reduz round-trips ao custo de granularidade de revogação — revogar durante um lease só tem efeito ao fim dele. Só faz sentido com muitos processors e latência intercontinental. Fora de escopo , mas boa resposta de palco.
Particionamento por mandato	Escala horizontal preservando a propriedade: cada mandato pertence a exatamente uma partição, com um líder. É como a v2 do registry cresce sem mudar semântica.
Consenso global (blockchain)	Resolve, e a latência inviabiliza. Ver §4.4.

4.14 Pergunta 14 — O que reutilizar em vez de criar protocolo proprietário

Regra de escopo: **a única coisa genuinamente nossa deve ser a DSL de política, o catálogo de reason codes e a máquina de estado do commit**. Tudo mais já existe.

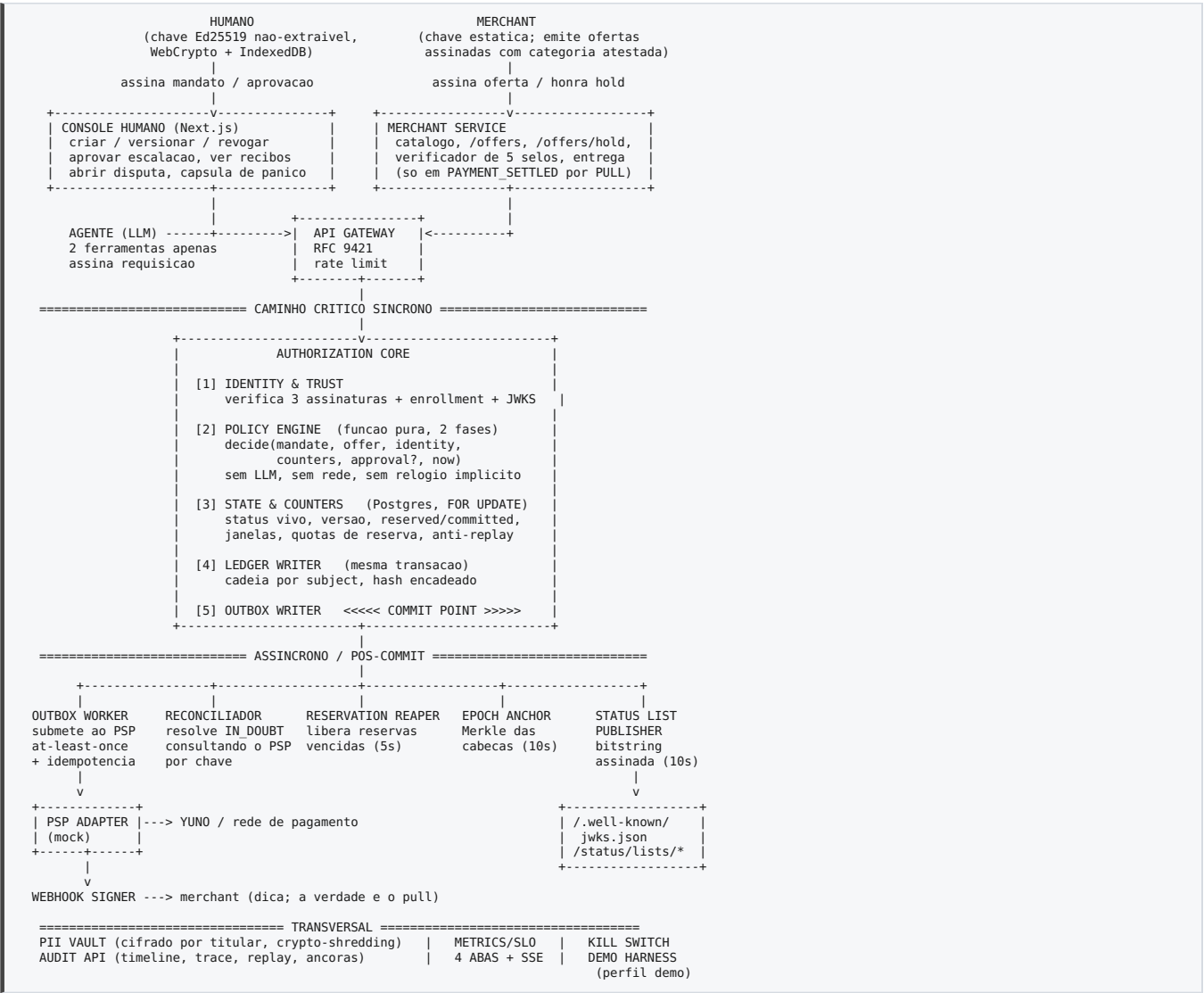
Padrão	Uso exato	Nota
W3C VC Data Model 2.0	Formato do mandato	Já é a base do AP2. Ganha-se <code>credentialStatus</code> , <code>validFrom/validUntil</code> e <code>proof</code> de graça.
W3C Bitstring Status List	Revogação e suspensão publicadas, com privacidade de rebanho	Resolve simultaneamente as perguntas 2, 4, 9 e 11. É o maior retorno por linha de código do documento.
AP2 (Intent / Cart / Payment Mandate)	Modelo de autoridade e vocabulário	A camada preenche a lacuna de revogação que o AP2 declara fora de escopo.
ACP	Formato de checkout	Adaptador mock compatível. Não reinventar checkout — a Yuno já cobre esse bloco.
RFC 7009 — OAuth Token Revocation	Semântica e forma do endpoint de revogação	Reuso subestimado: revogação de autoridade delegada é problema resolvido há uma década em OAuth. Copie a semântica, inclusive a idempotência do "revogar algo já revogado retorna 200".
RFC 7662 — Token Introspection	Forma do endpoint de status	"Esta autoridade ainda está viva, e com que escopo?" é literalmente introspection. Adotar o formato de resposta (<code>active</code> , <code>scope</code> , <code>exp</code> , <code>sub</code>) torna a camada legível para qualquer engenheiro de identidade em trinta segundos.
RFC 9396 — Rich Authorization Requests	Modelagem de escopo estruturado	O <code>authorization_details</code> do RAR é a forma canônica de expressar "pode gastar até X, nesta categoria, neste merchant". Alinhar os nomes de campo reduz a curva de adoção.
RFC 9421 — HTTP Message Signatures	Identidade do agente no ponto de uso	Base do Visa TAP. Usar o subconjunto declarado do §3.7, não um esquema próprio.
RFC 8785 (JCS) e RFC 8032 (Ed25519)	Canonicalização e assinatura	Já na v3.0. Manter.
RFC 9457 — Problem Details	Formato dos erros de API	Trivial e faz a API parecer profissional. <code>type</code> aponta para a documentação do reason code.
Idempotency-Key (IETF draft)	Cabeçalho de idempotência	Semântica idêntica à já especificada. Citar em vez de descrever.
Transactional Outbox + Saga	Commit point e compensação	Padrão maduro. Não é preciso justificar; é preciso apenas não improvisar no lugar dele.

A FRASE DE POSICIONAMENTO QUE SAI DESTA SEÇÃO

"Não estamos propondo criptografia nova nem protocolo novo. Estamos implementando a peça que o AP2 declara fora do seu escopo — revogação em tempo real — reutilizando VC Status List, OAuth Introspection e HTTP Message Signatures, e acrescentando a única coisa que ninguém padronizou ainda: **onde exatamente fica o commit point.**"

5. Arquitetura de implementação: visão geral

O que segue é a arquitetura v4 completa: a da v3.0 mais os componentes que as correções exigem (outbox, reconciliador, publicador de status list, JWKS, cofre de PII, quotas). O diagrama distingue explicitamente o que está no **caminho crítico síncrono** do que é assíncrono, porque essa distinção é o que a v3.0 não fazia e é a origem do achado C1.



5.1 Princípios de implementação, em ordem de prioridade

- 1. **Nada que chame a rede participa de uma transação de banco.** Se você precisar de um lock e de um round-trip HTTP na mesma função, o desenho está errado.
- 2. **Toda decisão é uma função pura de dados assinados.** `decide()` não recebe o objeto de requisição, não lê relógio, não abre socket. O tipo de entrada impede — e há um teste de propriedade que falha se alguém mudar a assinatura da função.
- 3. **Todo efeito econômico e seu registro nascem na mesma transação.** Não existe decisão sem evento de ledger, nem evento sem decisão.
- 4. **Toda operação que amplia poder exige assinatura do principal; toda que reduz, não.** Invariante I7 da v3.0, mantido.
- 5. **Falha desconhecida nunca libera recurso.** Preserva-se consumo até prova em contrário.
- 6. **Nenhum identificador interno atravessa a fronteira do merchant.** Handles opacos e pseudônimos pareados.

5.2 Estrutura do monorepo

/packages	
/schemas	tipos, JCS, assinatura/verificacao, ISO 4217, config de janelas + asserção de boot: token < receipt < offer, escalation < hold decide() puro, 2 fases, DSL de condicoes, vetores de conformidade
/policy	ZERO dependencia de I/O — nem tipos que a permitam
/core	transacoes, locks, contadores, janelas, ledger, outbox writer
/payments	maquina de estado de payment_attempt, outbox worker, reconciliador, adaptador PSP (interface + mock Yuno)
/identity	enrollment, JWKS, RFC 9421 (assinar e verificar), nonce cache
/status	publicador de bitstring status list + assinatura + validade
/privacy	pseudonimos pareados, handles opacos, arvore de divulgacao seletiva, cofre de PII (cifra por titular)
/merchant	catalogo, emissao de oferta, hold, verificador de 5 selos, receptor de webhook com verificacao de assinatura
/agent	loop do LLM (2 ferramentas) + modo determinístico + modo adversarial
/web	4 abas (humano, merchant, auditor, juiz) + SSE compartilhado
/demo-harness	relógio logico, reset, painel de ataques, seeds >>> so entra no bundle com BUILD_PROFILE=demo <<<
/scripts	reset.sh, attack-suite.sh, property-tests.sh, bench.sh, seed.sh
/vectors	vectors.json (conformidade) + fixtures assinadas

6. Modelo de dados completo

O que segue é o esquema da v3.0 com as correções aplicadas e as tabelas novas. Diferenças em relação à v3.0 estão marcadas.

```
-- ===== IDENTIDADE E CHAVES =====

CREATE TABLE principals (
  id          TEXT PRIMARY KEY,
  key_id      TEXT NOT NULL,
  public_key  TEXT NOT NULL,
  enrolled_at TIMESTAMPTZ NOT NULL,
  panic_commitment TEXT          -- [NOVO §4.3] SHA256(R), opcional
);

CREATE TABLE agents (
  id          TEXT PRIMARY KEY,
  principal_id TEXT NOT NULL REFERENCES principals(id),
  key_id      TEXT NOT NULL,
  public_key  TEXT NOT NULL,
  key_status  TEXT NOT NULL DEFAULT 'ACTIVE',    -- ACTIVE|REVOKED|COMPROMISED
  compromised_at TIMESTAMPTZ,
  enrollment  JSONB NOT NULL
);

CREATE TABLE layer_keys (
  kid          TEXT PRIMARY KEY,
  public_key   TEXT NOT NULL,
  status       TEXT NOT NULL,                  -- ACTIVE|RETIRED|COMPROMISED
  valid_from   TIMESTAMPTZ NOT NULL,
  valid_until   TIMESTAMPTZ,
  compromised_at TIMESTAMPTZ,
  root_signature TEXT NOT NULL                -- assinada pela raiz offline
);

-- ===== MANDATOS E CONTADORES =====

CREATE TABLE mandates (
  id          TEXT PRIMARY KEY,
  version     INT NOT NULL DEFAULT 1,
  status      TEXT NOT NULL,                  -- ACTIVE|SUSPENDED|REVOKED
  body        JSONB NOT NULL,
  signature   TEXT NOT NULL,
  status_index INT NOT NULL,                  -- [NOVO §4.2] posicao aleatoria
  status_list TEXT NOT NULL,                  -- [NOVO] id da lista
  updated_at  TIMESTAMPTZ NOT NULL
);
CREATE UNIQUE INDEX uq_status_slot ON mandates(status_list, status_index);

CREATE TABLE mandate_counters (
  mandate_id  TEXT NOT NULL REFERENCES mandates(id),
  window_key  TEXT NOT NULL,                  -- '2026-09'
  committed_minor BIGINT NOT NULL DEFAULT 0,  -- [MUDDOU M3] inteiro em unidade menor
  reserved_minor BIGINT NOT NULL DEFAULT 0,  -- [MUDDOU M3]
  uses_committed INT NOT NULL DEFAULT 0,
  uses_reserved INT NOT NULL DEFAULT 0,
  live_reservations INT NOT NULL DEFAULT 0,   -- [NOVO §3.6] grieving
  PRIMARY KEY (mandate_id, window_key)
);

CREATE TABLE revocations (
  id          TEXT PRIMARY KEY,
  mandate_id  TEXT NOT NULL REFERENCES mandates(id),
  kind        TEXT NOT NULL,                  -- REVOKE|SUSPEND|RESUME|REDUCE
  method      TEXT NOT NULL,                  -- SIGNED|SESSION|PANIC_CAPSULE
  principal_signature TEXT,                  -- presente se method=SIGNED
  panic_secret TEXT,                         -- presente se method=PANIC_CAPSULE
  requested_at TIMESTAMPTZ NOT NULL,
  effective_at TIMESTAMPTZ NOT NULL
);

-- ===== OFERTAS, DECISOES, PAGAMENTO =====

CREATE TABLE offer_sightings (
  merchant_id TEXT NOT NULL, nonce TEXT NOT NULL,
  offer_id TEXT NOT NULL, seen_at TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (merchant_id, nonce)
);

CREATE TABLE request_nonces (
  agent_id TEXT NOT NULL, nonce TEXT NOT NULL,
  seen_at TIMESTAMPTZ NOT NULL, expires_at TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (agent_id, nonce)
);

CREATE TABLE decisions (
  id          TEXT PRIMARY KEY,
  handle      TEXT NOT NULL UNIQUE,          -- [NOVO §3.5] opaco, vai ao merchant
  mandate_id  TEXT NOT NULL,
  mandate_version INT NOT NULL,
  pairwise_id TEXT NOT NULL,                  -- [NOVO §3.5] HMAC(mandate,merchant)
  offer_id    TEXT NOT NULL,
  offer_hash  TEXT NOT NULL,
  terms_hash  TEXT NOT NULL,
  transaction_hash TEXT NOT NULL,            -- [NOVO §4.6] vinculo do commit
  amount_minor BIGINT NOT NULL,
  currency    CHAR(3) NOT NULL,
  currency_exponent SMALLINT NOT NULL,      -- [NOVO M3]
  window_key  TEXT NOT NULL,
  decision    TEXT NOT NULL,                  -- ALLOW|DENY|ESCALATE
  reason_codes TEXT[] NOT NULL DEFAULT '{}',
  trace       JSONB NOT NULL,
  disclosure_root TEXT NOT NULL,             -- [NOVO §3.4] divulgacao seletiva
  nonce       TEXT NOT NULL,
  expires_at  TIMESTAMPTZ NOT NULL,
  consumed_at TIMESTAMPTZ,
  released_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_offer_live ON decisions(offer_id)
  WHERE decision='ALLOW' AND released_at IS NULL;
CREATE UNIQUE INDEX uq_decision_nonce ON decisions(nonce);
CREATE UNIQUE INDEX uq_tx_hash ON decisions(transaction_hash);

CREATE TABLE payment_attempts (
  id          TEXT PRIMARY KEY,
  decision_id TEXT NOT NULL UNIQUE REFERENCES decisions(id),
  state       TEXT NOT NULL,                  -- COMMITTED|SUBMITTED|SETTLED|
```

```

    psp_idempotency_key TEXT NOT NULL UNIQUE,          -- DECLINED|IN_DOUBT|COMPENSATED
    amount_minor        BIGINT NOT NULL,              -- = transaction_hash
    currency             CHAR(3) NOT NULL,
    psp_reference        TEXT,
    failure_code         TEXT,
    attempts            INT NOT NULL DEFAULT 0,
    committed_at        TIMESTAMPTZ NOT NULL,
    submitted_at        TIMESTAMPTZ,
    resolved_at         TIMESTAMPTZ
);
CREATE INDEX ix_attempts_unresolved ON payment_attempts(state, committed_at)
WHERE state IN ('COMMITTED','SUBMITTED','IN_DOUBT');

CREATE TABLE outbox (                                -- [NOVO $3.1] commit point
    id                 TEXT PRIMARY KEY,
    topic             TEXT NOT NULL,
    payload           JSONB NOT NULL,
    available_at      TIMESTAMPTZ NOT NULL,
    attempts         INT NOT NULL DEFAULT 0,
    locked_until      TIMESTAMPTZ,
    done_at          TIMESTAMPTZ
);
CREATE INDEX ix_outbox_ready ON outbox(topic, available_at) WHERE done_at IS NULL;

CREATE TABLE captures (
    agent_id          TEXT NOT NULL,
    idempotency_key   TEXT NOT NULL,
    decision_id       TEXT NOT NULL REFERENCES decisions(id),
    request_hash      TEXT NOT NULL,                  -- [NOVO] distingue retry de bug
    response_body     JSONB NOT NULL,
    status_code       INT NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL,
    PRIMARY KEY (agent_id, idempotency_key)
);

-- ===== ESCALACAO, LEDGER, WEBHOOK, PII =====

CREATE TABLE escalations (
    id TEXT PRIMARY KEY, decision_id TEXT NOT NULL, terms_hash TEXT NOT NULL,
    mandate_version INT NOT NULL, status TEXT NOT NULL, ttl_expires_at TIMESTAMPTZ NOT NULL,
    approval JSONB, approval_consumed_at TIMESTAMPTZ, hold_state TEXT
);

CREATE TABLE ledger (
    chain_key TEXT NOT NULL,
    seq       BIGINT NOT NULL,
    event_type TEXT NOT NULL,
    actor     JSONB NOT NULL,
    subject   JSONB NOT NULL,
    payload   JSONB NOT NULL,
    context   JSONB,
    occurred_at TIMESTAMPTZ NOT NULL,                -- texto do agente: NUNCA lido
    prev_hash  TEXT NOT NULL,
    hash       TEXT NOT NULL,
    PRIMARY KEY (chain_key, seq)
);

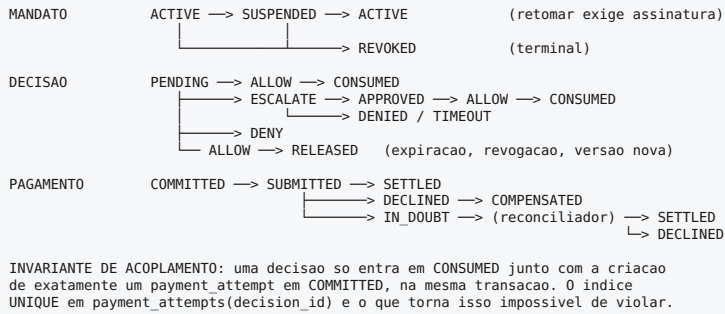
CREATE TABLE webhook_deliveries (                  -- [NOVO $3.2]
    id TEXT PRIMARY KEY, merchant_id TEXT NOT NULL, decision_handle TEXT NOT NULL,
    event_type TEXT NOT NULL, sequence INT NOT NULL, nonce TEXT NOT NULL,
    attempts INT NOT NULL DEFAULT 0, delivered_at TIMESTAMPTZ, last_status INT
);

CREATE TABLE pii_vault (                          -- [NOVO M2] crypto-shredding
    subject_id TEXT NOT NULL, field_hash TEXT NOT NULL,
    ciphertext BYTEA NOT NULL, PRIMARY KEY (subject_id, field_hash)
);
CREATE TABLE subject_keys (
    subject_id TEXT PRIMARY KEY, wrapped_key BYTEA, shredded_at TIMESTAMPTZ
);

```

7. Máquinas de estado e caminhos críticos

7.1 As três máquinas de estado, lado a lado



7.2 POST /authorize — caminho completo v4

```
1. verificar assinatura da requisicao (RFC 9421) + janela + nonce      [$3.7]
2. resolver agente: enrollment assinado, key_status = ACTIVE
3. verificar assinatura do merchant na oferta; kid conhecido no JWKS
4. carregar mandato; verificar assinatura do principal contra pubkey registrada
5. BEGIN
6.   SELECT * FROM mandates WHERE id=$m FOR UPDATE
7.   win := window_key(now, mandate.cumulative_window_tz)
8.   SELECT * FROM mandate_counters WHERE (m,win) FOR UPDATE -- cria se ausente
9.   INSERT INTO request_nonces -- conflito => DENY REQUEST_NONCE_REPLAY [NOVO]
10.  INSERT INTO offer_sightings -- conflito => DENY OFFER_NONCE_REPLAY
11.  se counters.live_reservations >= mandate.max_live_reservations
    => DENY RESERVATION_LIMIT [NOVO]
12.  decision := decide(mandate, offer, identity, counters, approval?, now)
    -- FASE A: gates duros, ordenada, primeira falha vence
    -- FASE B: avaliadores suaves, todos rodaram, agregacao
13.  se ALLOW:
    transaction_hash := sha256(decision_id||terms_hash||amount_minor||ccy)
    INSERT INTO decisions (... , handle, pairwise_id, disclosure_root, ...)
    UPDATE counters SET reserved_minor += a, uses_reserved += 1,
    live_reservations += 1
    se ESCALATE:
    INSERT decisions(ESCALATE) + escalations(ttl)
    reserva IGUAL a de ALLOW (senao duas escalacoes concorrem pelo saldo)
    pedir hold ao merchant FORA da transacao, depois do commit [$3.10-M5]
    se DENY:
    INSERT decisions(DENY) com todos os reason codes e trace
14.  INSERT INTO ledger (AUTHORIZATION_DECIDED) -- mesma transacao
15. COMMIT
16. montar recibo: visao do merchant (divulgacao seletiva) e visao do principal
17. emitir eventos SSE para as quatro abas
```

7.3 POST /capture — caminho completo v4

```
1. verificar assinatura da requisicao + Idempotency-Key
2. se (agent_id, idem_key) ja existe:
   se request_hash igual -> devolve resposta ORIGINAL, Idempotent-Replay: true
   se diferente -> 409 IDEMPOTENCY_KEY_REUSE
3. BEGIN
4.   SELECT * FROM mandates WHERE id=$m FOR UPDATE
5.   SELECT * FROM mandate_counters WHERE (m, decision.window_key) FOR UPDATE
6.   C1 decisao existe, ALLOW, consumed_at IS NULL, released_at IS NULL
   C2 now < decision.expires_at
   C3 decision.mandate_version == mandate.version
   C4 mandate.status == ACTIVE <<< a cena do demo
   C5 valid_from <= now <= valid_until
   C6 agente ACTIVE e assinatura do capture valida
   C7 terms_hash da oferta viva == decision.terms_hash (TERMS_DRIFT)
   C8 window_key(now) == decision.window_key (WINDOW_ROLLED)
   C9 FASE B inteira reexecutada com estado ATUAL
   C10 se houve escalacao: aprovacao valida, nao consumida, mesmos termos/versao
   C11 UPDATE decisions SET consumed_at=now() WHERE consumed_at IS NULL
   -- 0 linhas => DENY TOKEN_ALREADY_CONSUMED
7.   UPDATE counters: reserved -= a, committed += a,
   uses_reserved -= 1, uses_committed += 1,
   live_reservations -= 1
8.   INSERT INTO payment_attempts (state='COMMITTED', psp_idempotency_key=tx_hash)
9.   INSERT INTO outbox ('payment.submit') <<<<< COMMIT POINT >>>>>
10.  INSERT INTO captures (resposta original, request_hash)
11.  INSERT INTO ledger (CAPTURE_COMMITTED)
12. COMMIT
13. emitir ScopedPaymentToken (authorization proof, 60s, consume-once) [$4.12]
14. responder 202 { state: "COMMITTED", token, decision_handle }
   -- 202, nao 200: o dinheiro ainda nao se moveu, e a API nao deve mentir.
```

7.4 POST /mandates/{id}/revoke — três caminhos de entrada, uma transição

Entrada A: sessao autenticada (o caminho normal, mais facil possivel)
Entrada B: assinatura do principal (produz evidencia E9 para disputa)
Entrada C: capsula de panico (publica R; verifica SHA256(R)==commitment)

```
BEGIN
SELECT * FROM mandates WHERE id=$m FOR UPDATE -- mesmo lock do capture
UPDATE mandates SET status='REVOKED', updated_at=now()
UPDATE decisions SET released_at=now()
WHERE mandate_id=$m AND decision='ALLOW'
AND consumed_at IS NULL AND released_at IS NULL
-- devolver cada reserva ao SEU window_key, nao ao corrente [D7c da v3.0]
para cada decisao liberada:
UPDATE counters SET reserved_minor -= a, uses_reserved -= 1,
live_reservations -= 1
WHERE (mandate_id, window key = decisao.window key)
UPDATE escalations SET status='INVALIDATED' WHERE mandate_id=$m AND status='PENDING'
INSERT INTO revocations (kind, method, principal_signature|panic_secret, ...)
INSERT INTO ledger (MANDATE_REVOKED)
COMMIT
-- pos-commit: marcar o bit na proxima publicacao da status list;
-- emitir SSE; NAO tocar em payment_attempts ja COMMITTED.
```

O DETALHE QUE DECIDE O TRIAL BY FIRE

Repare na última linha: **a revogação não toca em payment_attempts que já estão em COMMITTED**. É a materialização do commit point. Se o juiz revogar durante o delay entre propõe e capture, o capture morre em C4. Se revogar depois do capture, a compra segue e a próxima falha. As duas coisas são corretas, são diferentes, e o sistema precisa explicar a diferença na tela — não no discurso.

8. Superfície de API v4

Delta sobre o §9.1 da v3.0. Rotas novas marcadas; rotas alteradas com o motivo.

Verbo	Rota	Autenticação	Nota
POST	/principals/enroll	Prova de posse	Aceita <code>panic_commitment</code> opcional. [ALTERADA §4.3]
POST	/agents/enroll	Assinatura do principal	—
POST	/agents/{id}/revoke-key	Sessão	Grava <code>compromised_at</code> : fronteira temporal da disputa.
POST	/mandates	Assinatura do principal	Sorteia <code>status_index</code> ; devolve <code>credentialStatus</code> . [ALTERADA §4.2]
PATCH	/mandates/{id}	Assinatura se amplia; sessão se restringe	Versão de destino dentro do payload assinado.
POST	/mandates/{id}/revoke	Sessão · assinatura · cápsula	Três entradas, uma transição. [ALTERADA §7.4]
POST	/mandates/{id}/panic	Conhecimento de R	[NOVA §4.3] Verifica <code>SHA256(R)</code> contra o commitment. Idempotente.
GET	/mandates/{id}/status	Sessão	Formato inspirado em RFC 7662: <code>{active, scope, exp, version}</code> . <code>no-store</code> .
POST	/authorize	Assinatura do agente (RFC 9421)	Nonce de requisição obrigatório. [ALTERADA §3.7]
POST	/capture	Assinatura + Idempotency-Key	Responde 202 com estado <code>COMMITTED</code> . [ALTERADA §7.3]
GET	/decisions/{handle}/status	Merchant autenticado	Por handle opaco . Inclui <code>PAYMENT_SETTLED</code> . É a fonte de verdade da entrega. [ALTERADA §3.2, §3.5]
GET	/decisions/{id}/trace	Principal ou auditor	Trace regra a regra; hash confere com o do recibo.
GET	/payments/{attempt_id}	Principal ou auditor	[NOVA §3.1] Estado da tentativa, tentativas, referência do PSP.
POST	/payments/{id}/reconcile	Operador	[NOVA §3.1] Força reconciliação de um attempt em dúvida.
POST	/offers, /offers/{id}/hold	Chave do merchant	Hold vira opcional ; equivalência de reemissão é o fallback. [ALTERADA M5]
GET	/escalations?status=pending	Sessão	—
POST	/escalations/{id}/resolve	Assinatura para APPROVE; sessão para DENY	Assina (<code>escalation_id</code> , <code>terms_hash</code> , <code>mandate_version</code> , veredito).
POST	/disputes · GET /disputes/{id}/verdict	Sessão	Agora com evidências E8 e E9. [ALTERADA]
GET	/ledger?subject= · POST /ledger/replay	Auditor	—
GET	/.well-known/jwks.json	Público	[NOVA §3.9] Inclui chaves aposentadas com janela de validade.
GET	/status/lists/{id}	Público	[NOVA §4.2] Bitstring assinada, <code>validFrom/validUntil</code> , privacidade de rebanho.
GET	/metrics	Interno	[NOVA M8] Prometheus.
POST	/demo/*	—	Só existe com BUILD_PROFILE=demo. <code>/demo/clock</code> , <code>/demo/reset</code> , <code>/demo/adversary</code> , <code>/demo/psp-outage</code> [NOVA] , <code>/demo/forge-webhook</code> [NOVA] .

9. MVA versus production-grade

A pesquisa pede as duas arquiteturas explicitamente. A diferença entre elas **não** é a semântica de segurança — essa é idêntica — e sim disponibilidade, escala, custódia e operação.

Dimensão	Arquitetura mínima viável (hackathon / piloto)	Production-grade
Autoridade de estado	Postgres único, <code>SELECT ... FOR UPDATE, READ COMMITTED</code> . Tudo no primário.	Postgres com réplica síncrona e failover automático, ou grupo Raft dedicado. Particionamento por <code>mandate_id</code> com um líder por partição. Leituras de autoridade nunca em réplica assíncrona.
Registry de revogação	Coluna <code>status</code> na tabela de mandatos, lida sob lock. Status list publicada a cada 10 s em arquivo estático.	Serviço dedicado com a mesma semântica; status list em CDN com assinatura destacada; âncoras Merkle publicadas em meio externo para irrefutabilidade contra o operador.
Commit point	Outbox na mesma tabela Postgres, worker in-process com <code>FOR UPDATE SKIP LOCKED</code> .	Outbox com CDC (Debezium) ou worker dedicado multi-réplica; fila durável; reconciliador com alarme e runbook; DLQ com revisão humana.
Chave do humano	Ed25519 não-extraível via WebCrypto em IndexedDB. Sem recuperação: re-enrollment revoga tudo.	Passkey/WebAuthn com atestação de dispositivo; multi-dispositivo com política de quórum; HSM ou custódia corporativa para principais empresariais; recuperação social ou administrativa com trilha assinada.
Chaves da camada e do merchant	Variável de ambiente, resolvidas por <code>kid</code> em chaveiro de demo; JWKS servido a partir dele.	KMS/HSM, rotação automatizada com sobreposição de 30 dias, JWKS assinado por raiz offline, cerimônia de rotação documentada.
Adaptador de pagamento	Mock que imprime a intenção no formato que a Yuno consumiria e devolve resultado simulado, com modo "derrubar PSP".	Integração real com a Yuno; suporte a autorização/captura separadas, parcial, reversal e refund; mapeamento de códigos do PSP para a máquina de estado; testes de caos.
Privacidade	Pseudônimos pareados, handles opacos, divulgação seletiva com Merkle, hashes no ledger.	Idem, mais cofre de PII em KMS com crypto-shredding, política de retenção automatizada, DPIA, e avaliação de PIR para a status list se a escala justificar.
Escalação	Console web com SSE e contador regressivo; TTL 120 s.	Push mobile, e-mail e SMS com deep link assinado; TTL 5–30 min; política de aprovador substituto para mandatos corporativos; fila com SLA.
Ledger	Tabela Postgres, cadeias por subject, âncoras em memória a cada 10 s.	Armazenamento WORM ou object storage com <i>object lock</i> ; âncoras publicadas externamente; verificação contínua em background com alarme.
Observabilidade	<code>/metrics</code> + rodapé ao vivo na aba do juiz.	Tracing distribuído, SLOs com <i>error budget</i> , alertas por <code>in_doubt_attempts</code> e <code>outbox_lag</code> , runbooks por sintoma.
Conformidade	Declarações em §26.3.	Escopo PCI documentado (fora, por não tocar PAN), avaliação LGPD, contratos de operador/controlador, retenção de 540 dias, política de resposta a incidentes.
Multi-processor	Autoridade única implícita.	<code>authorizationAuthority</code> nomeada no mandato; token consume-once como recurso exclusivo; leases opcionais para autonomia regional.

A PROPRIEDADE QUE SOBREVIVE INTACTA DOS DOIS LADOS

Em ambas as arquiteturas, a afirmação central é a mesma e é verificável: *um usuário pode retirar a autoridade de um agente até o instante em que uma transação específica é comprometida para submissão, e nenhum agente pode usar autorização revogada nem reutilizar autorização já consumida*. A MVA prova isso com um lock de linha em Postgres. A produção prova com o mesmo desenho e mais disponibilidade. **Nada na lista de diferenças altera a semântica de segurança** — e essa é a melhor resposta possível para "o que muda quando isso vira produto?".

10. Testes, SLOs e observabilidade

A v3.0 já tem a melhor estratégia de testes que um hackathon pode ter. A v4 acrescenta o que as correções exigem — e uma categoria nova que a v3.0 não tinha: **testes de falha injetada**, sem os quais a máquina de estado do pagamento é uma afirmação.

10.1 As cinco camadas

Camada	O que cobre	Critério de pronto
Unidade do motor	Um teste por reason code do Anexo A da v3.0 mais os 14 novos. Sem banco, sem rede.	48 testes, dois segundos, tudo verde em um comando.
Vetores de conformidade	Entrada completa e decisão esperada, em arquivo publicável.	Um terceiro reimplementa o motor e verifica contra o arquivo.
Propriedade	P1-P9 da v3.0 mais P10-P14 (§10.2).	10.000 sequências aleatórias, zero violações.
Concorrência	N requisições realmente paralelas, cem repetições.	Exatamente as que cabem no saldo passam. Um botão no console do juiz dispara vinte.
Falha injetada [NOVA]	PSP com timeout, PSP que recusa, PSP que responde duas vezes, banco que cai entre outbox e submissão, worker morto após submeter, relógio saltando para trás.	Nenhum cenário produz cobrança sem registro, registro sem cobrança, ou reserva liberada sem resposta definitiva.

10.2 As cinco propriedades novas

#	Propriedade	Protege
P10	Para toda decisão em CONSUMED existe exatamente um payment_attempt, e vice-versa.	Acoplamento decisão↔pagamento (§7.1). É a propriedade que impede cobrança fantasma.
P11	Nenhum payment_attempt em IN_DOUBT teve sua reserva liberada nem seu committed revertido.	A regra "fail-closed na autoridade, fail-conservador no contador" (§3.8).
P12	Em qualquer intercalação de revogação e capture, ou o capture falha com MANDATE_REVOKED, ou existe linha de outbox — nunca ambos, nunca nenhum.	A definição do commit point (§4.6). É a propriedade central do produto, escrita como teste.
P13	Nenhum campo de counters_snapshot, mandate_id ou principal_id aparece em nenhum payload destinado ao merchant.	Privacidade (§3.4, §3.5). Verificável por inspeção de serialização, com falha em CI.
P14	A soma de committed_minor por janela é igual à soma dos payment_attempts em SETTLED mais os em IN_DOUBT.	Conservação de valor entre o contador e o mundo real, incluindo a incerteza.

10.3 SLOs declarados

Indicador	Alvo	Como se mede
Latência p99 de /authorize	< 60 ms	Histograma no /metrics; exibido ao vivo na aba do juiz.
Latência p99 de /capture	< 80 ms	Idem. Não inclui a chamada ao PSP, que é assíncrona.
Overhead p99 no fluxo de pagamento	< 150 ms	Soma de authorize + status + capture.
Propagação de revogação p99	< 1 s	Do clique ao primeiro DENY consequente.
Attempts em IN_DOUBT por mais de 15 min	0	Alarme; é incidente por definição.
Divergências no replay do ledger	0	POST /ledger/replay, executado em CI e no palco.
Gasto autorizado fora do mandato	US\$ 0,00	Rodapé permanente. É a métrica do produto.

11. Plano de execução hora a hora

Premissa: **36 horas úteis, quatro pessoas**. Ajuste proporcional se o formato for diferente. A regra que organiza tudo: **uma fatia vertical fina e funcionando até H+12**, e só depois espessura. O erro clássico — e é o que o roadmap de doze blocos da v3.0 induz — é construir cada camada bem e integrar no fim.

11.1 Papéis

Pessoa	Domínio	Entregáveis
P1 — Núcleo	/schemas, /policy, /core	JCS e assinaturas; decide() de duas fases; contadores, locks e janelas; vetores de conformidade; testes de unidade e propriedade.
P2 — Dinheiro e trilha	/payments, ledger, disputa	Capture com revalidação completa; máquina de estado do pagamento; outbox e reconciliador; ledger encadeado, âncoras e replay; veredito de disputa.
P3 — Superfícies	/web, /identity	WebCrypto e enrollment; as quatro abas; SSE; divulgação seletiva na visão do merchant; console do juiz.
P4 — Mundo externo	/merchant, /agent, /demo-harness	Catálogo, emissão de oferta, hold, verificador de cinco selos, webhook assinado; agente LLM com duas ferramentas; agente adversarial; seeds e reset.

11.2 Cronograma

Janela	Marco	Conteúdo e critério de saída
H+0 → H+2	Fundação comum	Repositório, Docker Compose com Postgres, migrações, JCS, Ed25519, chaveiro, tipos compartilhados, asserção de ordem de janelas. Saída: todos os quatro conseguem rodar make dev e ver o serviço subir.
H+2 → H+8	Fatia vertical mais fina possível	P1: decide() só com gates A0-A11 e avaliadores B1-B6. P2: /authorize e /capture com lock e contadores, sem outbox ainda. P3: uma tela só, com WebCrypto e assinatura de mandato. P4: catálogo estático e emissão de oferta assinada. Saída: mandato assinado no navegador → oferta → ALLOW → capture → linha no ledger. Sem beleza.
H+8 → H+12	Revogação funcionando	Revogação sob o mesmo lock, liberação de reservas por window_key, C1-C11 completos no capture. Saída (portão obrigatório): revogar durante o delay faz o capture morrer com MANDATE_REVOKED. Se isso não estiver pronto em H+12, corte escopo, não qualidade.
H+12 → H+17	Commit point e pagamento	P2 isolado nisto: payment_attempts, outbox, worker, reconciliador, adaptador mock com modo de queda. P1: fase B completa, DSL de condições, banda de três faixas. Saída: derrubar o PSP no meio do capture deixa a compra em "confirmando" e a reserva presa; religar resolve.
H+17 → H+22	Escalação e as quatro abas	P1/P2: banda de escalação, hold, aprovação assinada, TTL, invalidação por versão. P3: quatro abas com SSE. P4: agente LLM completo com o delay encenado. Saída: voo de US\$ 300 escala, é aprovado com assinatura e captura; mudar o limite no meio mata a aprovação.
H+22 → H+26	Correções de privacidade e webhook	P3: handles opacos, pseudônimos pareados, divulgação seletiva. P4: webhook assinado + verificação no merchant + botão de forjar. P1: quotas de reserva e nonce de requisição. Saída: o recibo do merchant não contém o orçamento; forjar liquidação não entrega mercadoria.
H+26 → H+30	Auditoria e disputa	P2: âncoras, replay determinístico, sete (agora nove) evidências, veredito, exportação. P1: testes de propriedade P1-P14. Saída: /ledger/replay devolve divergent: 0; contestar a compra produz veredito com a regra citada.
H+30 → H+33	Console do juiz e adversário	P4: um botão por linha da tabela de ameaças, relógio, reset, agente adversarial com dezessete caminhos, catálogo B2B. P3: rodapé de métricas ao vivo. Saída: reset em menos de dois segundos; adversário roda em trinta.
H+33 → H+36	Ensaio	Três ensaios cronometrados. Um deles com alguém de fora operando o Anexo C. Congelamento de código em H+35. Saída: quem não escreveu o código consegue quebrar — ou não conseguir quebrar — sozinho.

CRITÉRIOS DE CORTE REVISADOS

Se em **H+17** o commit point não estiver funcionando, corte nesta ordem, sem discussão: *catálogo B2B → testes de propriedade → âncoras de época → tela de disputa → agente adversarial → verificação de cadeia → divulgação seletiva (cai para dois recibos assinados separados)*. **Nunca se corta:** revalidação completa no capture · lock dos contadores · chave do humano no dispositivo · máquina de estado do pagamento · webhook não sendo fonte de verdade. As duas primeiras porque o juiz vai testar; a terceira porque ele vai perguntar; as duas últimas porque são as que separam este projeto dos outros.

12. Demo v4, riscos e planos B

12.1 Cinco minutos, revisados

Tempo	Cena	O que acontece
0:00-0:20	O problema	Uma frase: hoje o merchant bloqueia o agente e perde a venda, ou o trata como humano e come a fraude. As quatro abas já visíveis.
0:20-0:55	Marta cria o mandato	Frase em linguagem natural vira objeto estruturado; ela lê o objeto e assina no navegador. "Chave gerada aqui, não-extraível, nunca enviada." Nenhum número de cartão em lugar nenhum.
0:55-1:35	A compra feliz	Preço cai para US\$ 130; oferta assinada; ALLOW; capture; o estado passa por "confirmando" antes de "liquidado" , e isso é dito em cinco palavras: <i>o dinheiro tem um antes e um depois</i> .
1:35-2:15	Fora do mandato	US\$ 300 → ESCALATE com contador; Marta aprova assinando. US\$ 900 → DENY sem botão de aprovar. "Nunca aprovado em silêncio."
2:15-3:15	Trial by fire	O juiz assume. Revogação durante o delay → MANDATE_REVOKED. Se houver tempo: baixar o limite (STALE_VERSION), dois cliques rápidos (fatiamento concorrente), replay do token.
3:15-3:45	Queda do PSP [NOVA]	O juiz derruba o processador durante o capture. Tela do humano: "pagamento em confirmação". Contador continua reservado. Nada é entregue. Religa: reconcilia em segundos. Frase: "sem resposta não é o mesmo que recusa, e tratar as duas coisas igual é como se perde dinheiro de verdade."
3:45-4:10	Forjar liquidação [NOVA]	Um clique dispara um webhook falso de PAYMENT_SETTLED para o merchant. A mercadoria não sai; o evento aparece no ledger como assinatura inválida. Frase: "o webhook avisa; quem manda é a consulta."
4:10-4:35	O adversário	Um clique: dezessete caminhos, zero sucessos, três segundos, <i>gasto autorizado fora do mandato: US\$ 0,00</i> .
4:35-5:00	Disputa + horizontalidade + pitch	Marta contesta; nove evidências e o veredito com a regra citada. Troca de catálogo para B2B. Fecha com a frase de venda.

SE O TEMPO CAIR PARA TRÊS MINUTOS

Corte, nesta ordem: disputa, horizontalidade, adversário, escalação de US\$ 900. **Preserve sempre:** criação do mandato com assinatura no dispositivo, revogação durante o delay, e queda do PSP. As três respondem, respectivamente, "quem tem a autoridade", "ela pode ser retirada" e "o que acontece quando o mundo real falha" — e a terceira é a que nenhum outro time terá.

12.2 Riscos novos e planos B

Risco	Sinal	Plano B
Máquina de estado do pagamento não fica pronta	H+17 sem outbox	Adaptador síncrono com timeout curto e estado IN_DOUBT apenas registrado, sem reconciliador. Perde-se a cena da queda do PSP; mantém-se a correção conceitual e diz-se em uma frase o que falta.
Divulgação seletiva atrasa o front	H+26 sem árvore Merkle	Dois recibos assinados separadamente, ambos com decision_id e terms_hash. Resolve o vazamento em quinze minutos; declare a simplificação.
Merchant sem endpoint de hold	—	Equivalência de reemissão (\$3.10-M5). Já é o desenho; o hold vira otimização.
Juiz pergunta por PSP real ou certificação ACP	Juiz técnico da Yuno	Resposta curta e honesta: mock deliberado, ponto de integração desenhado, e a máquina de estado do pagamento é exatamente o que torna a integração real uma troca de adaptador.
Juiz pergunta "e a privacidade disso tudo?"	Juiz que pensou bem	§3.5 e §4.11: pseudônimos pareados, handles opacos, privacidade de rebanho, minimização — e o limite declarado, que é o mais convincente dos cinco.
Juiz aponta que o relógio controlável é trapaça	Provável	§3.3: pacote separado, não compilado em produção, e todo salto no ledger.

Anexo A — Reason codes novos

Acrescentam-se catorze códigos aos trinta e quatro da v3.0. Cada linha é também um caso de teste.

Código	Fase	Severidade	Parte	Significado
REQUEST_SIGNATURE_STALE	pré-A0	DENY	Agente	created/expires fora da janela de 60 s ou skew além de 2 s.
REQUEST_NONCE_REPLAY	pré-A0	DENY	Agente	Nonce de requisição já visto para este agente.
REQUEST_DIGEST_MISMATCH	pré-A0	DENY	Agente	Content-Digest não confere com o corpo recebido.
RESERVATION_LIMIT	B (pré)	DENY	Agente	Reservas vivas no teto. Defesa contra griefing de orçamento.
UNDECLARED_ATTRIBUTE	B3	ESCALATE	—	Oferta traz atributo não previsto e o escopo está em modo strict.
OFFER_CLOCK_SKEW	A3	DENY	Merchant	issued_at da oferta no futuro além da tolerância.
REVOCATION_UNVERIFIABLE	A8	DENY	Camada	Estado de revogação indisponível. Fail-closed na autoridade.
CURRENCY_EXPONENT_MISMATCH	A10	DENY	Emissor	Expoente declarado diverge do ISO 4217 para a moeda.
PAYMENT_DECLINED	pós-commit	—	Emissor	Recusa determinística do PSP. Dispara compensação.
PAYMENT_IN_DOUBT	pós-commit	—	—	Sem resposta definitiva. Contador permanece consumido; reconciliação em curso.
COMPENSATION_APPLIED	pós-commit	—	—	Reserva e uso devolvidos após recusa definitiva.
WEBHOOK_SIGNATURE_INVALID	merchant	DENY	Atacante	Notificação de liquidação sem assinatura válida. Não libera mercadoria.
WEBHOOK_REPLAY	merchant	DENY	Atacante	Nonce ou event_id de webhook repetido.
PANIC_COMMITMENT_MISMATCH	revog.	DENY	—	SHA256(R) não confere com o commitment do mandato.

Códigos de API que não são decisões do motor: `PRINCIPAL_SIGNATURE_REQUIRED` (401), `VERSION_CONFLICT` (409), `IDEMPOTENCY_KEY_REUSE` (409), `PRINCIPAL_KEY_NOT_HELD_BY_SERVER` (403), `DEMO_ENDPOINT_NOT_AVAILABLE` (404, perfil prod).

Anexo B — Vetores de conformidade novos

```
{
  "vectors": [
    {
      "id": "V041-commit-point-revocation-loses",
      "note": "revogacao chega DEPOIS do insert no outbox",
      "sequence": ["authorize(130)", "capture()", "revoke()"],
      "expect": {
        "capture": "COMMITTED",
        "outbox_rows": 1,
        "mandate_status": "REVOKED",
        "next_authorize": "DENY"
      }
    },
    {
      "id": "V042-commit-point-revocation-wins",
      "sequence": ["authorize(130)", "revoke()", "capture()"],
      "expect": {
        "capture": "DENY",
        "reason_codes": ["MANDATE_REVOKED"],
        "outbox_rows": 0,
        "reserved_after": 0
      }
    },
    {
      "id": "V043-psp-timeout-preserves-consumption",
      "sequence": ["authorize(130)", "capture()", "psp_timeout()"],
      "expect": {
        "attempt_state": "IN DOUBT",
        "committed_minor": 13000,
        "reserved_minor": 0,
        "compensated": false
      }
    },
    {
      "id": "V044-psp-decline-compensates",
      "sequence": ["authorize(130)", "capture()", "psp_decline()"],
      "expect": {
        "attempt_state": "DECLINED",
        "committed_minor": 0,
        "uses_committed": 0,
        "ledger_has": "COMPENSATION_APPLIED"
      }
    },
    {
      "id": "V045-reservation-griefing-blocked",
      "sequence": ["authorize(130)", "authorize(130)"],
      "expect": {
        "second": "DENY",
        "reason_codes": ["RESERVATION_LIMIT"]
      }
    },
    {
      "id": "V046-merchant-view-has-no-counters",
      "note": "P13: privacidade verificada por serializacao",
      "expect": {
        "merchant_payload_contains": [],
        "forbidden_fields": ["counters_snapshot", "mandate_id", "principal_id", "cumulative_limit"]
      }
    },
    {
      "id": "V047-request-nonce-replay",
      "sequence": ["authorize(req A)", "authorize(req A)"],
      "expect": {
        "second": "DENY",
        "reason_codes": ["REQUEST_NONCE_REPLAY"]
      }
    },
    {
      "id": "V048-status-list-staleness",
      "note": "verificador que aceita lista mais velha que o contrato falha",
      "list_age_seconds": 45,
      "max_status_staleness_seconds": 10,
      "expect": {
        "verification": "REJECT"
      }
    },
    {
      "id": "V049-panic-capsule-revokes",
      "sequence": ["panic(R)"],
      "expect": {
        "mandate_status": "REVOKED",
        "method": "PANIC_CAPSULE",
        "ledger_has": "MANDATE_REVOKED"
      }
    },
    {
      "id": "V050-strict-attribute-escalates",
      "offer": "fixtures/offer-flight-130-first-class.json",
      "mandate_attribute_policy": "strict",
      "expect": {
        "outcome": "ESCALATE",
        "reason_codes": ["UNDECLARED_ATTRIBUTE"]
      }
    }
  ]
}
```

Anexo C — Menu do juiz v4 (linhas novas)

Somam-se às vinte e quatro linhas do Anexo C da v3.0. Cada uma tem um botão no console.

☐ Teste você mesmo	Como fazer	O que deve acontecer
☐ Derrubar o processador	Botão "PSP offline" durante o capture	Estado "em confirmação", reserva mantida, nada entregue. Religar resolve sozinho.
☐ Recusa do processador	Botão "PSP recusa"	PAYMENT_DECLINED, compensação aplicada, orçamento devolvido, evento no ledger.
☐ Forjar liquidação	Botão "enviar webhook falso"	Mercadoria não sai; WEBHOOK_SIGNATURE_INVALID.
☐ Reenviar liquidação verdadeira	Botão "duplicar webhook"	Um único efeito; WEBHOOK_REPLAY no segundo.
☐ Prender o orçamento	Botão "10 autorizações sem capture"	Uma reserva; nove RESERVATION_LIMIT; alerta na tela do humano.
☐ Reenviar requisição capturada	Botão "replay da requisição assinada"	REQUEST_NONCE_REPLAY.
☐ Cápsula de pânico	Colar o segredo R impresso	Mandato revogado sem usar o dispositivo com a chave.
☐ Ver o que o merchant vê	Botão "payload do merchant"	Sem orçamento, sem mandate_id, sem principal_id — e a assinatura ainda verifica.
☐ Correlacionar dois merchants	Botão "comparar identificadores"	Nenhum identificador em comum entre as duas lojas.
☐ Tentar acelerar o relógio em produção	Botão "modo prod + /demo/clock"	404 DEMO_ENDPOINT_NOT_AVAILABLE: o código não está no bundle.
☐ Ver o custo em milissegundos	Rodapé de métricas	p99 de /authorize abaixo de 60 ms, ao vivo, durante os ataques.

Anexo D — Checklist contra o enunciado, atualizado

Requisito do case	Onde é atendido	Estado após a v4
Humano cria mandato sem entregar o cartão cru	\$8.1 e \$8.5 da v3.0	Mantido. Nenhum PAN em nenhum artefato; referência de cofre apenas.
Merchant verifica antes de aceitar	\$8.4 v3.0 + \$3.2 v4	Reforçado: cinco selos, entrega por pull, webhook assinado e não-autoritativo.
Compra ponta a ponta com registro para o humano	\$6 v3.0 + \$3.1 v4	Reforçado: o registro agora distingue autorizado, em confirmação e liquidado.
Fora do mandato: rejeitado ou escalado, nunca em silêncio	\$10.3 B4, \$13	Mantido. Banda de três faixas.
Revogação ao vivo funciona	\$14, \$15 v3.0 + \$4.5 v4	Reforçado: commit point formal e prova informal de que não há race.
Agente impersonado	\$7.2, A0 + \$3.7 v4	Reforçado: nonce e janela na própria requisição assinada.
Trilha auditável legível por humano, merchant e auditor	\$8.7, \$16, \$20	Mantido, mais eventos de pagamento e de revogação.
Trial by fire operável pelos juízes	\$17, \$20, \$25.3 + Anexo C v4	Reforçado: onze ataques novos, incluindo queda do PSP e webhook forjado.
Quem responde pela disputa	\$19.2 + E8 e E9	Reforçado: nove evidências; nova linha para entrega baseada só em webhook.
Bônus: fluxo completo de disputa	\$19	Mantido.
Bônus: condições ricas	\$10.4, \$11, \$12.3 + M4	Reforçado: required_attributes e modo strict.
Bônus: defesa contra agente adversarial	\$18 + \$3.6 v4	Reforçado: dezessete caminhos; griefing de orçamento incluído.

Anexo E — Fontes e nota de revalidação

As referências [Y1]-[Y7] e [P1]-[P8] da v3.0 permanecem válidas e não são reproduzidas aqui. Acrescentam-se as verificações feitas para esta revisão.

Ref	Fonte	Uso e estado
[V1]	Lançamento da plataforma Agentic Commerce da Yuno	Reconfirmado em 29/08/2026. Anunciada em 21 de janeiro de 2026, dá aos merchants um caminho direto para capturar receita de experiências de compra e recomendação conduzidas por IA, com painel ao vivo de pedidos, taxas de aprovação, funis de conversão e atribuição de receita, e está disponível tanto para clientes quanto para não clientes da Yuno. Sustenta \$2 da v3.0.
[V2]	Página de produto Agentic Commerce da Yuno	Descreve infraestrutura pronta para o Agentic Commerce Protocol, com sincronização de catálogo, endpoints certificados e verificações de conformidade gerenciadas. Confirma que o bloco de checkout está coberto e que nossa camada se posiciona antes dele.
[V3]	Análise pública das limitações do AP2	A fonte mais importante desta revisão. Confirma que o AP2 v0.2 é de abril de 2026 e que não há mecanismo de revogação dentro do protocolo, ficando a cargo do credential provider ou da carteira. Sustenta \$4.1 inteiro.
[V4]	Especificação AP2 — mandatos como credenciais verificáveis	Confirma a estrutura de Intent, Cart/Checkout e Payment Mandate, com estágios open e closed, e que o Payment Mandate é compartilhado com rede e emissor para sinalizar envolvimento de agente. Sustenta \$4.2 e \$4.12.
[V5]	W3C Bitstring Status List	Base do desenho de revogação publicada com privacidade de rebanho (§4.4, §4.11). Herdado de [P5] da v3.0, agora com uso específico.
[V6]	RFC 7009 (revogação) e RFC 7662 (introspection)	Reuso de semântica para os endpoints de revogação e status (§4.14).

Nota: antes de citar qualquer data ou título em slide, reconfirmar na página. Apresentar para a Yuno com uma data errada sobre a própria Yuno custa caro.

A FRASE DE FECHAMENTO, REVISADA

"A Yuno torna o comércio agentic possível. Nossa camada torna a autoridade desse agente verificável — **e revogável até o último instante em que isso ainda significa alguma coisa.**"

O AP2 declara que não resolve revogação e delega para fora de si. É essa peça que estamos entregando: não criptografia nova, não protocolo novo, mas a definição precisa de onde fica o commit point, com a prova de que antes dele o usuário manda e depois dele o sistema de pagamentos assume.